GAME FINAL / SOURCE-BASED DESIGN REVIEW

Design patterns and SOLID in your code

Exact C# excerpts, original line numbers, and practical explanations

Unity 6000.0.32f1 | Working-tree snapshot: 02 November 2026

388
project scripts reviewed

53
verbatim code excerpts

5
SOLID principles explained

A guided reading of the actual implementation: what the syntax means, how the objects collaborate, why the design helps, and where the same design is only partial.

Scope: your gameplay and custom tooling. Imported vendor code and generated input source are excluded. No gameplay files were modified.

Static source analysis. Build, Play Mode, and scene reachability are not certified.

Contents

Use the PDF bookmarks or click an entry below. Code excerpts retain the original source line numbers.

01. How to read this review 3

02. Project architecture at a glance 4

03. A C# syntax guide using your code 5

04. Singleton: convenience and hidden dependencies 7

05. State: character behavior and nested interruptions 9

06. Command and Template Method: executable story assets 11

07. Strategy: composable NPC pacing and transition conditions 14

08. Observer: event channels, dialogue notifications and cleanup 16

09. Adapter and Facade: separating the chess engine 18

10. Object Pool, bounded reuse and caches 20

11. Snapshots, undo records and cloning 22

12. Data-driven configuration and component composition 25

13. Flyweight-style tree geometry and spatial partitioning 27

14. S - Single Responsibility Principle 29

15. O - Open/Closed Principle 31

16. L - Liskov Substitution Principle 33

17. I - Interface Segregation Principle 35

18. D - Dependency Inversion Principle 37

19. Subsystem findings beyond the main patterns 39

20. Focused improvements and verification limits 42

21. Complete script coverage 44

01. How to read this review

Start with the excerpt, follow the caller, then assess the design claim.

This document explains design patterns and SOLID through the current Game Final code. Each worked example connects three things: the exact C# declaration, the place that uses it, and the resulting behavior. The goal is to make the design visible in source rather than attach pattern names to filenames.

The review covers 388 project C# files: 357 under `Final/Assets/0_Scripts`, 19 under `Final/Assets/_TreeLeap`, six under `Final/Assets/Pesonal_Assets`, four custom scripts directly under `Final/Assets/Editor`, and the standalone `FollowTarget.cs` and `HouseShadowDistanceZone.cs`. The complete inventory appears at the end. Imported asset libraries, package source, shader programs, and the generated `InputSystem_Actions.cs` wrapper are outside this architectural assessment.

What an excerpt means

- Every code panel is extracted directly from a current source file. The original spelling, identifiers, comments, capitalization, and source line numbers are retained. These are selected contiguous ranges, not rewritten examples.
- An excerpt can show only part of a class or method. Its caption and path identify the context; it is not necessarily a standalone compilable program.
- Long source lines wrap visually. A `>` in the number gutter marks a continuation of the preceding source line. It is a document marker, not a C# operator added to the code.
- Paths are relative to the project folder. Source line numbers can move after an edit. The document describes the working-tree snapshot on 02 November 2026, including existing local changes.
- Syntax coloring is a reading aid. A color does not determine a token's semantics. The explanations identify the relevant declarations, calls, and contracts explicitly.

How evidence is classified

Assessment	Meaning in this document
Demonstrated	The abstraction and its consumer are present; source shows how the collaboration is used.
Partial / pattern-like	The mechanism resembles a pattern, but important structural or encapsulation features differ. The qualification is explained.
Design risk	The code contains a concrete lifecycle, ownership, contract, or coupling concern. Runtime failure is not assumed unless it was reproduced.
Not demonstrated	A name, interface declaration, empty scaffold, or commented-out code is insufficient evidence that an architecture uses a pattern.

A pattern and a principle answer different questions. A pattern names a recurring collaboration, such as State or Observer. A principle helps evaluate responsibilities, extension, contracts, interface size, or dependency direction. A Singleton can be a real pattern and still weaken dependency inversion. A short helper can follow good design without implementing any named GoF pattern.

02. Project architecture at a glance

The strongest boundaries separate reusable policy, runtime state, and Unity presentation.

The project combines reusable C# cores with Unity components that coordinate scene objects. Its most convincing modularity appears where a consumer uses a small contract and where authored definitions are separated from per-run state. The table is a navigation map, not a rating of every class in a folder.

Area	Main collaboration	Design assessment
Characters and cameras	State objects, transition predicates, player interruption stack	Good local separation; registration and lifecycle ownership need care.
Story	StoryAction assets -> StoryNodeRunner -> concrete actions	Command plus Template Method; global manager lookups retain coupling.
NPC walking	Path geometry + follower + speed-modifier pipeline	Strong Strategy, interface segregation, and extension boundaries.
Vehicle and input	ICarInput -> vehicle physics; player adapter or intro cruise	Real runtime replacement with stable physics consumer.
Interaction	IInteractable and IPlacementTarget -> UI and placement consumers	Useful capabilities; mutable prompt/availability contracts are inconsistent.
Chess	Unity-facing facade/adapter -> engine position, move generation, evaluation and search	Strong domain/presentation separation; some mutable internals escape.
Phone and minigames	App polymorphism, view helpers, shared input and events	Useful composition; large app/game coordinators mix responsibilities.
TreeLeap	Shared trunk profiles + per-tree shapes + spatial grid + rig/effects helpers	Defensible Flyweight-style sharing and spatial partitioning; creature partial files remain one class.
Ambience and effects	Baked geometry, reused buffers, source-owned suppression and visual/audio modules	Good performance and cleanup practices; mostly concrete Unity dependencies.
Persistence and loading	Explicit checkpoint DTO, backup writes, world restoration, IProgress reporting	Clear data format and progress boundary; save orchestration knows many concrete systems.

A useful reading route

Read the syntax guide first if C# abstractions are still unfamiliar. Then follow State, Command/Template Method, and Strategy for behavioral collaboration. Read Adapter/Facade and shared geometry for structural boundaries. The SOLID chapters revisit the same code as design evidence, including counterexamples. The final subsystem chapter covers smaller components and unfinished scaffolds that do not warrant a named pattern.

Do not infer a pattern from the name

`ChessEngineBridge` performs adaptation, not the GoF Bridge collaboration. `ICookable` is a class despite its prefix. A method named Build is not automatically the GoF Builder pattern. A registry that finds an existing object is not a factory. Coroutine syntax alone establishes neither Command nor State.

03. A C# syntax guide using your code

Recognize the language features that make the later pattern examples work.

The same few C# features appear throughout the design. Understanding the declaration is only half the story: the consumer must actually use it. An interface that nobody consumes does not invert a dependency, and inheritance alone does not prove that subclasses preserve a behavioral contract.

E01 | An interface declares the controls a consumer can request

`Final/Assets/0_Scripts/Vehicle/ICarInput.cs`
`Original lines 1-13`

```
/// <summary>
/// The five controls the SUV understands. Values are 0..1 so keyboard keys read as
/// on/off while gamepad triggers and sticks stay analogue.
/// Implement this to drive the car from a cutscene or a scripted intro sequence.
/// </summary>
public interface ICarInput
{
    float Accelerate { get; }
    float SteerLeft { get; }
    float SteerRight { get; }
    float Reverse { get; }
    float Brake { get; }
}
```

`interface ICarInput` defines a contract without choosing a device or algorithm. `float Accelerate { get; }` is a read-only property in that contract: callers can read it, but cannot assign it through an `ICarInput` reference. An implementation may still use `private set` internally. The five properties describe driving rather than every possible player input, which makes this a focused capability.

E02 | An abstract base class combines a common algorithm with a required subclass operation

`Final/Assets/0_Scripts/Story/RootStoryAction/StoryAction.cs`
`Original lines 12-27`

```
    public abstract class StoryAction : ScriptableObject
    {
        public float Delay = 0f;

        public IEnumerator Run()
        {
            if (Delay > 0f)
                yield return StorySkip.Wait(Delay);

            // Execute even when the delay was skipped. The delay is pacing; the body is usually
            // world state - a teleport, a progress stage, an animator change - and a skipped beat
            // that never applied its state leaves the scene wrong for everything after it.
            yield return Execute();
        }

        protected abstract IEnumerator Execute();
```

`abstract class` prevents direct construction of an incomplete StoryAction. The colon in `: ScriptableObject` declares its base type. `protected abstract IEnumerator Execute()` requires a subclass implementation and limits direct access to the inheritance family. The public `Run()` method is concrete: it already defines the shared sequence. `yield return` lets the coroutine suspend and resume; when Unity executes this coroutine through the runner, the yielded child enumerator participates in that sequence.

E03 | Generic value comparison, event invocation, and listener registration

Final/Assets/0_Scripts/DayNight/Observable.cs
Original lines 26-47

```
26      public void Set(T value)
27      {
28          if (EqualityComparer<T>.Default.Equals(this.value, value))
29              return;
30          this.value = value;
31          Invoke();
32      }
33
34      public void Invoke()
35      {
36          ValueChanged?.Invoke(value);
37      }
38
39      public void AddListener(Action<T> handler)
40      {
41          ValueChanged += handler;
42      }
43
44      public void RemoveListener(Action<T> handler)
45      {
46          ValueChanged -= handler;
47      }
```

`T` is a generic type parameter: the same Observable implementation can hold different value types. `EqualityComparer<T>.Default` supplies the appropriate equality comparison. `ValueChanged?.Invoke(value)` invokes the event only when listeners exist. `+= handler` subscribes a delegate; `-= handler` removes the same delegate. A lambda such as `() => condition` packages an expression as callable behavior. Events let a producer publish a change without naming each consumer.

E04 | Readonly value data and expression-bodied properties with safe short-circuiting

Final/Assets/0_Scripts/PlayerInput/InputButton.cs
Original lines 15-38

```
15  public readonly struct InputButton
16  {
17      private readonly InputAction primary;
18      private readonly InputAction secondary;
19
20      public InputButton(InputAction primary, InputAction secondary = null)
21      {
22          this.primary = primary;
23          this.secondary = secondary;
24      }
25
26      /// <summary>True only on the frame the button went down. Use this for one-shot actions.</summary>
27      public bool Pressed =>
28          (primary != null && primary.WasPressedThisFrame()) ||
29          (secondary != null && secondary.WasPressedThisFrame());
30
31      /// <summary>True on every frame the button is down, first frame included.</summary>
32      public bool Held =>
33          (primary != null && primary.IsPressed()) ||
34          (secondary != null && secondary.IsPressed());
35
36      /// <summary>True only on the frame the button came back up.</summary>
37      public bool Released =>
38          (primary != null && primary.WasReleasedThisFrame()) ||
```

`readonly struct` makes InputButton a value type whose instance fields cannot be reassigned after construction. `private readonly InputAction primary` keeps the reference fixed; it does not make the referenced InputAction

object immutable. The => after a property defines its getter as an expression. && short-circuits, so the method on primary is not called when primary is null. || combines two possible bindings. A default InputButton therefore returns false safely. This is null-object-like behavior, not a separate polymorphic Null Object class.

Syntax	How to read it in the project
public / private / protected / internal	Controls who can access a member; useful boundaries protect invariants and limit coupling.
virtual / override	A base supplies replaceable behavior; a derived type provides its implementation. Check the caller to confirm polymorphism is used.
new Type(dependency)	Constructs an object and passes a collaborator. This is injection, but DIP additionally depends on which type the consumer references.
in PlayerStateProfile profile	Passes the struct by read-only reference. It expresses a read-only argument view; it does not freeze objects referenced by the struct.
is IInitializable<IPlayerInput>	Tests whether an object offers a capability and binds a typed reference so the caller can invoke that contract.
[SerializeField] / [CreateAssetMenu]	Unity authoring instructions: serialize a field or expose asset creation. They do not by themselves imply a GoF pattern.
partial class	Places pieces of one class in separate files. The pieces still share state and responsibility.
HashSet<T> / Dictionary<K,V>	Collections used for membership or keyed lookup. Their use alone does not establish Observer, Flyweight, or a factory.

Exact source versus teaching explanation

All code panels show the implementation as it exists. Suggested changes later in the document are described in prose and explicitly treated as recommendations. They are not mixed into the quoted source.

04. Singleton: convenience and hidden dependencies

A real pattern whose usefulness depends on lifetime and ownership discipline.

Many managers and databases expose a static Instance. Some inherit the generic Singleton<T>; others implement their own assignment, duplicate handling, and teardown. The pattern gives callers a common access point to an intended single service. It does not establish that every implementation enforces uniqueness safely or that global access is the best dependency boundary.

E05 | Lazy global lookup and automatic construction through a generic base

Final/Assets/0_Scripts/Utilities/Singleton.cs
Original lines 3-19

```
3  public class Singleton<T> : MonoBehaviour where T : Component
4  {
5      private static T instance;
6      public static T Instance
7      {
8          get
9          {
10             if (instance == null)
11             {
12                 instance = (T)FindFirstObjectByType(typeof(T));
13                 if (instance == null)
14                 {
15                     SetupInstance();
16                 }
17             }
18             return instance;
19         }
```

`Singleton<T>` is generic, and `where T : Component` restricts its type argument to Unity components. `private static T instance` is shared for the closed generic type, rather than stored separately on each object. The static Instance getter first searches the scene, then calls SetupInstance when no component is found. A call such as `NpcDatabase.Instance` therefore resolves a concrete global object, and can also trigger creation. The caller does not declare that dependency in its constructor or initialization contract.

This is convenient for a central database or service, and the shared base removes repeated access code. The costs are implicit setup requirements, lifetime coupling, and harder substitution in tests. The presence of a field called Instance is not enough to claim robust Singleton behavior: several project classes log duplicates but still overwrite their static reference, whereas others destroy duplicates or clear their reference on destruction.

E06 | Duplicate removal currently does not distinguish the existing owner from itself

Final/Assets/0_Scripts/Utilities/Singleton.cs
Original lines 43-54

```
43     private void RemoveDuplicates()
44     {
45         if (instance == null)
46         {
47             instance = this as T;
48             //DontDestroyOnLoad(gameObject);
49         }
50         else
51         {
52             Destroy(gameObject);
53         }
54     }
```

The branch checks only whether instance is null. If the Instance getter finds a scene component before that component's Awake runs, instance already refers to that same component when RemoveDuplicates is reached. The else branch then schedules its own GameObject for destruction. This is a source-level lifecycle risk; no Play Mode reproduction was performed. A consistent implementation needs an explicit same-owner check and a clear policy for duplicate objects, destruction, scene transitions, and disabled domain reload.

The existing HasInstance accessor is a useful distinction: callers can ask whether a service exists without triggering construction. That matters during cleanup, where accidentally creating a new service while the scene is unloading would violate ownership expectations. Such details make a larger difference to reliability than the pattern name itself.

> **SOLID connection**
>
> The main concern is D: consumers bind to concrete global services. A focused interface passed into a reusable consumer makes its dependency visible and replaceable. A scene-level composition root can still resolve the one real service and inject it; every class does not need its own service locator.

05. State: character behavior and nested interruptions

One lifecycle contract, with different containers for different gameplay needs.

The shared `StateMachine` is a clear implementation of the State pattern. It holds the current behavior as `IState`, whose contract contains `OnEnter()`, `OnExit()` and `Update()`. Concrete classes such as `IdleState`, `MoveState` and `BearSleepState` supply the behavior. The machine controls the lifecycle; it does not contain the details of sitting, navigating or playing a bear animation. This separates the mechanism for changing behavior from the behavior itself.

E07 | State replacement: validate the request, exit the previous state, publish the next state, then enter it.

`Final/Assets/0_Scripts/StateMachine/StateMachine.cs`
`Original lines 19-40`

```
19      public void ChangeState(IState State)
20      {
21          if (State == null) return;
22
23          // This used to compare an IState against a StateNode. C# allows it - StateNode is
24          // not sealed, so the compiler cannot rule out a subclass implementing IState - but
25          // it is always false at runtime, so the guard never once fired and re-entering the
26          // current state silently ran OnExit followed by OnEnter.
27          if (CurrentState != null && ReferenceEquals(CurrentState.State, State)) return;
28
29          var PrevState = CurrentState?.State;
30          var NextNode = GetOrAddNode(State);
31
32          PrevState?.OnExit();
33
34          // Assigned BEFORE OnEnter, so the machine reports the state it is actually in while
35          // that state is starting up, and a re-entrant ChangeState from inside OnEnter is not
36          // overwritten by this assignment afterwards.
37          CurrentState = NextNode;
38
39          NextNode.State?.OnEnter();
40      }
```

In this excerpt, `IState State` means the parameter can be any object implementing the interface. `ReferenceEquals` compares object identity and prevents restarting the same instance. The `?.` operator calls a method only when its receiver is non-null. Assigning `CurrentState` before `OnEnter()` means entry code sees the newly selected state, including when entry triggers another transition. The local `var` declarations infer concrete variable types; they do not remove compile-time type checking.

E08 | The NPC composition point creates concrete states and connects them to mode predicates.

Final/Assets/0_Scripts/NPC/NpcController.cs
Original lines 47-65

```
47          var BaseState = new BaseState(this, Animator);
48          var SitState = new SitState(this, Animator);
49          var WaitForPlayerState = new WaitForPlayerState(this, Animator);
50          var IdleState = new IdleState(this, Animator);
51          var MoveState = new MoveState(this, Animator);
52          var InterruptState = new InterruptState(this, Animator);
53          var SitDiningState = new SitDiningState(this, Animator,DinningChair);
54          SitDining = SitDiningState;
55          // Define transitions
56          Any(BaseState, new FuncPredicate(() => StateData.StateMode == NpcMode.None));
57          Any(SitState, new FuncPredicate(() => StateData.StateMode == NpcMode.Sitting));
58          Any(IdleState, new FuncPredicate(() => StateData.StateMode == NpcMode.Idle));
59          Any(MoveState, new FuncPredicate(() => StateData.StateMode == NpcMode.Moving));
60          Any(InterruptState, new FuncPredicate(() => StateData.StateMode == NpcMode.Interrupt));
61          Any(WaitForPlayerState, new FuncPredicate(() => StateData.StateMode == NpcMode.WaitForPlayer))
  >  ;
62          Any(SitDiningState, new FuncPredicate(() => StateData.StateMode == NpcMode.SitDining));
63
64          // Set initial state
65          stateMachine.SetState(BaseState);
```

Here, `new SitState(this, Animator)` injects the actor and animator into a behavior object. `new FuncPredicate(() => StateData.StateMode == NpcMode.Sitting)` packages a deferred boolean test: the lambda is evaluated later, rather than freezing its current result. `Any` registers a condition applicable from any current state. When another system requests Sitting, the next machine update can select that behavior. `TalkState` and `StandState` exist elsewhere but are not registered by this setup, so their presence alone does not demonstrate an active transition.

Bear AI reuses the same engine through `BearStateMachine.SetupStateMachine`. It constructs sleep, wake, walk, advance, charge and kill states. That reuse supports Single Responsibility and Open/Closed: a new behavior can implement the contract without rewriting lifecycle dispatch. Dependency Inversion is local and meaningful because dispatch depends on `IState` and predicates. Liskov substitution still requires each implementation to leave the actor in a valid condition; interface conformance cannot guarantee sensible cleanup.

E09 | Player interruptions use explicit stack restoration instead of the shared transition graph.

Final/Assets/0_Scripts/Player/StateMachinePlayer/PlayerStateStack.cs
Original lines 131-142

```
131     public void Pop()
132     {
133         if (frames.Count <= 1)
134             return;
135
136         int top = frames.Count - 1;
137         SuspendFrame(top);
138         frames.RemoveAt(top);
139
140         DiscardStaleFrames();
141         ResumeTop();
142     }
```

The player has different requirements. `PlayerStateStack.Push` suspends the top frame, while `Pop` removes an interruption, discards stale payloads and resumes the surviving frame. For example, dialogue entered while seated should return to sitting. `frames.Count <= 1` protects the permanent base state. Effects are applied through `IPlayerStateEffects`, so the container need not manipulate a camera directly. This is stack-based state management, not the predicate-driven graph above.

E10 | TreeLeap dispatches enum modes with a switch: an FSM, but not the polymorphic State implementation.

`Final/Assets/_TreeLeap/Scripts/TreeLeapCreature.cs`
`Original lines 262-278`

```
262             switch (_mode)
263             {
264                 case Mode.Ground: UpdateGround(dt); break;
265                 case Mode.Jump: UpdateJump(dt); break;
266                 case Mode.Tree: UpdateTree(dt); break;
267                 case Mode.StandUp: UpdateStandUp(dt); break;
268             }
269             // analytic phases set the rotation directly; keep the angular velocity for the springs th
  > at follow
270             if (_mode == Mode.Jump) _bodyAngVel = TLMath.ToRotVec(_bodyRot * Quaternion.Inverse(_prevB
  > odyRot)) / dt;
271             UpdateAttach();
272
273             _procW = Mathf.MoveTowards(_procW, _procWGoal, _procWRate * dt);
274             float w = TLMath.Smooth01(_procW);
275             if (_mode != Mode.Ground || _procW > 0f)
276             {
277                 UpdateLimbs(dt);
278                 UpdateSpine(dt);
```

Limitation

TreeLeap's Ground, Jump, Tree and StandUp branches remain methods on one partial class. Multiple source files do not create separate responsibilities. The shared graph also exposes mutable collections, keys nodes by state type, and has no explicit transition priority. BearAggresiveAdvanceState starts a coroutine on entry without cancelling it on exit. BearStateMachine additionally derives from MonoBehaviour despite being constructed with new. These limit encapsulation and lifecycle reliability; recognizing the pattern is not a claim that every implementation is robust.

06. Command and Template Method: executable story assets

Story operations become configurable objects, while one base method owns their shared execution sequence.

The story system uses two complementary patterns. Command packages an operation and its parameters inside an object; Template Method fixes an algorithm's common sequence while subclasses provide one step. `StoryAction` provides both roles. Each concrete action is an authored ScriptableObject asset, and `StoryNode.actions` stores a list typed as the base class. The runner can invoke those objects without knowing whether one changes progression, moves an NPC, updates the journal or presents a choice.

E11 | Run owns the template: optional pacing delay followed by the subclass operation.

Final/Assets/0_Scripts/Story/RootStoryAction/StoryAction.cs
Original lines 12-28

```
12      public abstract class StoryAction : ScriptableObject
13      {
14          public float Delay = 0f;
15
16          public IEnumerator Run()
17          {
18              if (Delay > 0f)
19                  yield return StorySkip.Wait(Delay);
20
21              // Execute even when the delay was skipped. The delay is pacing; the body is usually
22              // world state - a teleport, a progress stage, an animator change - and a skipped beat
23              // that never applied its state leaves the scene wrong for everything after it.
24              yield return Execute();
25          }
26
27          protected abstract IEnumerator Execute();
28      }
```

`abstract class` prevents creating an incomplete base action directly. `Run()` is a concrete, non-virtual method, so callers share this delay-then-execute sequence. `protected abstract IEnumerator Execute()` requires a subclass implementation while keeping that implementation outside the public calling surface. `IEnumerator` and `yield return` allow Unity's coroutine scheduler to suspend and resume execution. A coroutine alone is not Command; the polymorphic action object, stored operation data and uniform invocation establish that pattern here.

The delay uses `StorySkip.Wait`, which is a custom skippable wait. Importantly, a skipped delay does not bypass `Execute()`. An action that teleports an actor or advances progression still needs to apply its world change after pacing is shortened. That distinction separates presentation timing from game-state mutation. The template reduces duplicated timing code and provides a single place to alter this common policy. It does not automatically make the action reversible or safe to execute twice.

E12 | A node starts its action coroutines together and waits for completion or a node skip.

Final/Assets/0_Scripts/Story/RootStoryAction/StoryNodeRunner.cs
Original lines 278-301

```
278             int completedActions = 0;
279             int totalActions = 0;
280
281             foreach (StoryAction action in node.actions)
282             {
283                 if (action == null)
284                     continue;
285
286                 totalActions++;
287
288                 // The count is a local, not a field on the node. StoryNode is a shared asset:
289                 // writing progress into it dirtied the .asset file and left the last run's numbers
290                 // committed to the project.
291                 StartCoroutine(RunSingleAction(action, () => completedActions++));
292             }
293
294             // Read after the actions start, so only a node skip pressed during THIS node counts.
295             // A plain line skip is deliberately not watched here: it collapses the waits inside the
296             // actions and lets them finish by themselves, which is what makes a conversation
297             // advance a line at a time instead of ending outright.
298             int nodeGeneration = StorySkip.NodeGeneration;
299
300             yield return new WaitUntil(() =>
301                 completedActions >= totalActions || StorySkip.NodeGeneration != nodeGeneration);
```

The `foreach (StoryAction action in node.actions)` loop handles every compatible subclass uniformly. `StartCoroutine` launches each action through `RunSingleAction`, whose body yields `action.Run()` and then invokes its completion callback. The lambda `() => completedActions++` captures the counter belonging to this node run. `new WaitUntil(() => ...)` repeatedly tests whether the actions finished or the skip generation changed. Actions therefore run together, not one after another merely because they appear in a list.

Both counters are local variables, rather than fields on the shared StoryNode asset. Two runs do not overwrite one shared completion count, and runtime progress does not dirty an authored asset. This is a useful separation between reusable configuration and execution state. For a concrete scenario, one node can start dialogue, a camera cut and a progression operation together, with different action delays expressing their timing. Completion waits for each action's own implementation to return.

E13 | A concrete command stores its requested stage and supplies only the operation body.

`Final/Assets/0_Scripts/Story/OtherStory/SetGameProgressStageAction.cs`
`Original lines 6-15`

```
6       [CreateAssetMenu(menuName = "StoryAction/Set Game Progress Stage")]
7       public class SetGameProgressStageAction : StoryAction
8       {
9           public GameProgressStage ProgressStage;
10
11          protected override IEnumerator Execute()
12          {
13              GameProgressManager.Instance.SetProgressStage(ProgressStage);
14              yield break;
15          }
```

`SetGameProgressStageAction : StoryAction` declares inheritance, and `override` supplies the required execution step. `ProgressStage` is the command's serialized parameter. The manager call performs the operation, and `yield break` ends this iterator immediately. A designer can create another asset with another stage without changing the runner. Adding a genuinely new operation normally means adding another subclass. These are strong local Open/Closed and responsibility-separation examples.

Limitation

The runner depends on the StoryAction abstraction, but many commands still locate concrete services through Instance, including this progression action. That limits Dependency Inversion and isolated testing. Node skipping allows already-started action coroutines to continue, so old work can overlap a later node. Some actions also launch additional work and return before that work ends. Completion and cancellation semantics therefore require care. There is no undo contract, and StoryNode is not a strict Composite because it does not implement StoryAction itself.

07. Strategy: composable NPC pacing and transition conditions

Small behavior contracts keep reusable mechanisms independent of individual policies.

NPC path pacing is one of the project's strongest extensibility examples. `NPCPathFollower` owns following a path, while `IPathSpeedModifier` supplies a policy for adjusting pace. Waiting for a companion, reacting to a marker and matching a group do not each need a special branch inside the follower. Different objects implement the same operation and are invoked through that abstraction. This is Strategy-style behavior composition, with multiple strategies applied as a pipeline rather than selecting exactly one.

E14 | The contract has one job: accept the current scale and return the adjusted scale.

`Final/Assets/0_Scripts/NPC/IPathSpeedModifier.cs`
`Original lines 7-14`

```
7   public interface IPathSpeedModifier
8   {
9       /// <summary>
10      /// Takes the speed scale so far (1 is normal walking pace, 0 is standing still) and
11      /// returns the adjusted one.
12      /// </summary>
13      float ModifySpeedScale(float scale, NPCPathFollower follower);
14  }
```

`interface` declares an operation without choosing its implementation. `float ModifySpeedScale(float scale, NPCPathFollower follower)` returns a number rather than directly assigning an agent's speed. The first parameter carries the result calculated so far; the second supplies context such as current path progress. The narrow operation is a good Interface Segregation example. Implementers are not required to implement navigation, animation or UI methods. However, the concrete follower parameter still exposes a broader dependency than a minimal read-only context would.

E15 | The follower composes discovered components, a per-path marker runner and external pacing policies.

`Final/Assets/0_Scripts/NPC/NpcPathFollower.cs`
`Original lines 303-321`

```
303     private float EvaluateSpeedScale()
304     {
305         float scale = 1f;
306
307         foreach (IPathSpeedModifier modifier in modifiers)
308         {
309             if (modifier is Behaviour behaviour && !behaviour.isActiveAndEnabled)
310                 continue;
311
312             scale = modifier.ModifySpeedScale(scale, this);
313         }
314
315         if (markerRunner != null)
316             scale = markerRunner.ModifySpeedScale(scale, this);
317
318         for (int i = 0; i < externalModifiers.Count; i++)
319             scale = externalModifiers[i].ModifySpeedScale(scale, this);
320
321         return Mathf.Max(0f, scale);
```

`float scale = 1f` establishes normal pace. Each assignment `scale = modifier.ModifySpeedScale(scale, this)` feeds one result into the next call. The C# pattern `modifier is Behaviour behaviour` checks whether this policy is also a Unity component and binds a typed variable. The following enabled-state test skips inactive components. The marker runner and externally registered policies then participate, and `Mathf.Max(0f, scale)` prevents a negative final scale.

This is not a strict Chain of Responsibility: there is no generic handled flag that stops later policies from running.

`PathWaitForTarget` implements the contract to ease off or hold when a target falls behind. `PathMarkers.Runner` implements pauses and speed zones. `PathGroup.MemberPacer` adjusts formation pace and is installed with `AddSpeedModifier`. A plausible combined scenario is an NPC walking with a group while approaching a slow zone and waiting for the player. The same evaluation mechanism composes all three concerns. Interactions still deserve verification: a later policy receives the earlier result, and the interface alone does not prevent it from replacing that result.

The marker runner also demonstrates sound ownership. A path stores authored markers, while a new runner stores one follower's fired flags, remaining pause and waiting state. Two walkers sharing a path therefore do not share one set of completion flags. The follower is open to new policy implementations, which supports Open/Closed; its evaluation code depends on `IPathSpeedModifier`, which supports Dependency Inversion. Group pacing, path geometry and movement execution have distinct responsibilities.

E16 | A delegate-backed predicate adapts a boolean function to the transition-condition interface.

`Final/Assets/0_Scripts/StateMachine/FuncPredicate.cs`
`Original lines 1-10`

```
using System;
public class FuncPredicate : IPredicate
{
    Func<bool> condition { get; }
    public FuncPredicate(Func<bool> condition)
    {
        this.condition = condition;
    }
    public bool Evaluate() => condition.Invoke();
}
```

Transition conditions use a related small strategy. `Func<bool>` means a callable function with no parameters that returns bool. The constructor stores it in the get-only `condition` property; `this.condition` distinguishes that property from the parameter. The expression-bodied `Evaluate() => condition.Invoke()` forwards the interface call to the supplied function. NPC setup can pass a lambda reading its current mode, while the state machine only calls `IPredicate.Evaluate()`. No NPC-specific condition is required inside the engine.

Limitation

These are local abstraction boundaries, not proof that all dependencies are inverted. Individual pacing policies still use concrete controllers and global Database lookups. The follower establishes a particular policy order, so conflicting rules need defined semantics. A new state or mode also needs composition and registration changes even though its predicate can reuse the same adapter. Do not label the pipeline a Decorator: policies receive a scale, rather than wrapping another modifier behind the same interface.

08. Observer: event channels, dialogue notifications and cleanup

Publish an occurrence without embedding every response inside the publisher.

The Observer pattern appears in both ScriptableObject event channels and ordinary C# events. A publisher maintains or exposes subscriptions; interested listeners register their response. The useful separation is that the source of an occurrence does not need to implement every resulting behavior. In this project, a trigger can raise a shared GameEvent asset while scene listeners connect that asset to inspector-configured UnityEvent responses. This is a concrete event-channel architecture, not evidence of a general-purpose message broker or Mediator.

E17 | The ScriptableObject channel synchronously notifies every currently registered listener.

`Final/Assets/0_Scripts/GameState/GameEvent.cs`
`Original lines 17-24`

```
17      public void Raise()
18      {
19          Debug.Log("Number of listeners found: " + listeners.Count);
20          foreach (GameEventListener listener in listeners)
21          {
22              listener.OnEventRaised();
23          }
24      }
```

`Raise()` loops over `listeners`, a `HashSet<GameEventListener>`. A set prevents duplicate entries for the same listener, but it does not express an authored response priority. Calling `listener.OnEventRaised()` is synchronous: ordinary method execution proceeds through this loop. The event does not automatically create a coroutine, background thread or delayed message. `GameEventTrigger.OnTriggerEnter` is one publisher; its player-tag check and one-shot flag determine when it calls `GameEvent.Raise()` without specifying the scene responses.

E18 | Unity component lifetime controls subscription, while UnityEvent supplies the authored response.

`Final/Assets/0_Scripts/GameState/GameEventListener.cs`
`Original lines 10-24`

```
10      private void OnEnable()
11      {
12          gameEvent.RegisterListener(this);
13      }
14
15      private void OnDisable()
16      {
17          gameEvent.UnRegisterListener(this);
18      }
19
20      public void OnEventRaised()
21      {
22          Debug.Log("Event raised");
23          response.Invoke();
24      }
```

`OnEnable()` registers this component, and `OnDisable()` unregisters it. `this` identifies the observer instance. When notified, `response.Invoke()` executes whatever response was configured in the inspector. For example, a channel raised by entering an area could be connected to a door response and a UI response without putting either implementation in the trigger. Adding a compatible listener extends the reactions while leaving the publisher unchanged, supporting local Open/Closed and separation of responsibilities.

E19 | Dialogue announces the line independently of the subtitle UI and then starts its voice.

Final/Assets/0_Scripts/Story/RootStoryAction/StoryDialogue.cs
Original lines 49-66

```
49          public static void Speak(DialogueEntry entry, Transform speaker)
50          {
51              if (entry == null)
52                  return;
53
54              LineStarted?.Invoke(entry);
55
56              if (entry.soundID == SoundID.Invalid)
57              {
58                  Debug.LogWarning($"Story dialogue line \"{Preview(entry.text)}\" has no sound clip ass
 >  igned.");
59                  return;
60              }
61
62              // Registered before it plays, so a skip on the very next frame can still silence it.
63              StorySkip.RegisterVoice(entry.soundID);
64
65              PlayVoice(entry.soundID, speaker);
66          }
```

`StoryDialogue` uses `public static event Action<DialogueEntry> LineStarted`. `Action<DialogueEntry>` describes callbacks receiving one entry and returning nothing. The `event` keyword restricts invocation to the declaring type; outside code subscribes using `+=` and removes its callback using `-=`. In `Speak`, `LineStarted?.Invoke(entry)` safely does nothing when there are no subscribers. The notification occurs before audio validation, so a text line can still be announced even when its sound clip is missing. The skip system also records voices before playback.

E20 | StoryContext listens for dialogue and skip notifications, then releases both subscriptions when disabled.

Final/Assets/0_Scripts/Story/StoryAddons/StoryContext.cs
Original lines 30-42

```
30      void OnEnable()
31      {
32          StorySkip.Skipped += OnStorySkipped;
33          StoryDialogue.LineStarted += OnLineStarted;
34      }
35
36      void OnDisable()
37      {
38          // Both are static events. A listener that does not detach here outlives the scene it
39          // belongs to and writes into a destroyed TMP_Text the next time one fires.
40          StorySkip.Skipped -= OnStorySkipped;
41          StoryDialogue.LineStarted -= OnLineStarted;
42      }
```

`OnLineStarted` translates the entry into a subtitle update; the dialogue utility does not directly reference a TMP text object. This is useful Single Responsibility separation and avoids three independent playback implementations drifting apart. `StorySkip.Skipped` lets the same context clear stale presentation when a beat is skipped. Because these events are static, their subscriptions can otherwise outlive a scene component. Matched subscription and removal are therefore a practical ownership rule, and the static systems also reset their events during subsystem registration.

Limitation

The GameEvent listener collection is public and Raise enumerates it directly. A response that disables a listener can unregister during that enumeration, risking collection-modification failures. A snapshot or deliberately deferred subscription changes would make this boundary safer. The C# event examples protect invocation better than a public callback field, but still depend on correct cleanup. Observer reduces direct response coupling; it does not automatically prove Dependency Inversion, thread safety, deterministic response ordering or exception isolation. Scene wiring and actual runtime outcomes were not verified by this source review.

09. Adapter and Facade: separating the chess engine

Translate representations at the boundary, then expose a simpler game-facing API.

The chess system is one of the clearest examples of layered design in this project. Scene-facing code uses ChessPiece, ChessMove and Unity's Vector2Int coordinates. Engine code uses integer square indices, bitboards and packed ushort moves. ChessEngineBridge translates between those representations. BoardState then presents a smaller game-facing API over Position. These are useful Adapter and Facade roles, respectively. The class name ChessEngineBridge should not be taken as evidence of the GoF Bridge pattern: this code does not define independently extensible abstraction and implementation hierarchies.

E21 | Adapter: translate a packed engine move into the scene-facing ChessMove value.

`Final/Assets/0_Scripts/ChessMinigame/ChessEngineBridge.cs`
`Original lines 77-90`

```
77      public static ChessMove ToChessMove(ushort engineMove)
78      {
79          if (engineMove == EngineMove.None)
80              return default;
81
82          int promotion = EngineMove.PromotionType(engineMove);
83
84          return new ChessMove(
85              ToCoordinates(EngineMove.From(engineMove)),
86              ToCoordinates(EngineMove.To(engineMove)),
87              promotion >= 0 ? ToPieceType(promotion) : ChessPieceType.None,
88              EngineMove.IsCastle(engineMove),
89              EngineMove.IsEnPassant(engineMove));
90      }
```

In this excerpt, static means conversion does not require a component instance. The ushort parameter is the engine's compact move representation. EngineMove.None is a sentinel; returning default produces an empty ChessMove rather than attempting conversion. EngineMove.From and To extract packed square values, and ToCoordinates maps those values to board coordinates. The conditional expression promotion >= 0 ? ... : ... maps the engine's promotion sentinel into ChessPieceType.None. Named conversion methods make the representation change explicit instead of scattering bit manipulation throughout input and rendering code.

E22 | Facade: callers request a legal move and receive a structured result.

Final/Assets/0_Scripts/ChessMinigame/BoardState.cs
Original lines 163-172

```
163     public bool TryApplyMove(ChessMove requestedMove, out ChessMoveResult result)
164     {
165         result = default;
166
167         if (!ChessEngineBridge.TryToEngineMove(position, requestedMove, out ushort engineMove))
168             return false;
169
170         result = ApplyEngineMove(engineMove);
171         return true;
172     }
```

TryApplyMove combines conversion, legal-move validation and application behind one operation. Its bool return tells the caller whether the request succeeded; out ChessMoveResult carries the resulting capture, castling, promotion and check information. Setting result = default ensures the output is assigned even on failure. The early return prevents ApplyEngineMove from seeing a rejected request. This gives ChessGameManager and the input layer a clear boundary: request a move, then react to its result. They do not need to understand Position's internal mailbox, hash key or undo stack.

E23 | MVC-like separation: the view observes board and move changes and releases its subscriptions.

Final/Assets/0_Scripts/ChessMinigame/ChessBoardView.cs
Original lines 42-58

```
42     private void OnEnable()
43     {
44         if (gameManager == null)
45             return;
46
47         gameManager.OnBoardChanged += RenderBoard;
48         gameManager.OnMoveApplied += HandleMoveApplied;
49     }
50
51     private void OnDisable()
52     {
53         if (gameManager == null)
54             return;
55
56         gameManager.OnBoardChanged -= RenderBoard;
57         gameManager.OnMoveApplied -= HandleMoveApplied;
58     }
```

- Actual flow: player input identifies a square, ChessGameManager calls BoardState, BoardState validates through the adapter, and the manager publishes move events. ChessBoardView moves visual pieces; CapturedPiecesDisplay updates its tray independently.
- Example: an en passant capture removes a piece from a different square than the destination. ChessMoveResult carries CapturedSquare, so the view can render this without independently reimplementing the chess rule.
- SOLID connection: domain rules, translation and presentation have distinct reasons to change, supporting SRP. Engine classes have no Unity namespace imports, making their logic easier to exercise independently of scenes.
- The += and -= operations attach and detach the same method groups. Matching lifecycle cleanup prevents a disabled view from continuing to respond to events and avoids retaining stale subscribers.

Limits of the pattern

This is MVC-like separation, not strict MVC or full dependency inversion. ChessGameManager still calls concrete ChessUI methods, and BoardState publicly exposes its mutable EnginePosition. That escape hatch lets callers bypass the facade. An adapter centralizes translation; it does not automatically make the underlying engine replaceable through an interface.

10. Object Pool, bounded reuse and caches

Reuse expensive objects and previous work, while keeping the different mechanisms distinct.

The project uses several forms of reuse, but their purposes differ. AudioManager uses Unity's ObjectPool<AudioSource>, which is an actual object pool: clients borrow an object and return it for later use. CameraApp cycles through preallocated textures and sprites in a ring buffer. The chess engine caches evaluated search positions in a transposition table. All three can reduce runtime cost, but a cache stores previous answers, whereas a pool reuses objects. Neither mechanism should automatically be called Flyweight, whose defining idea is shared intrinsic state among many logical objects.

E24 | Object Pool: borrow an AudioSource, configure it, play it, reset it and release it.

`Final/Assets/0_Scripts/GameState/AudioManager.cs`
`Original lines 50-63`

```
50      private IEnumerator PlayAtPosition(AudioClip clip, Vector3 position)
51      {
52          AudioSource instance = AudioSourcePool.Get();
53          instance.spatialBlend = 1;
54          instance.transform.position = position;
55          float length = clip.length;
56          instance.clip = clip;
57          instance.Play();
58          yield return new WaitForSeconds(length);
59          instance.Stop();
60          instance.spatialBlend = 0;
61          instance.clip = null;
62          AudioSourcePool.Release(instance);
63      }
```

AudioSourcePool.Get() borrows an available source, or lets the pool create one when necessary. The pool is initialized in OnEnable with CreateAudioSource passed as a creation delegate. The coroutine's IEnumerator return type allows yield return new WaitForSeconds(length) to suspend playback coordination without blocking the main thread. Before Release(instance), the source is stopped and its clip is cleared. That reset matters because the next borrower receives the same object. For a door slam followed later by a distant branch sound, the source object can be reused at a new position rather than repeatedly constructing and destroying GameObjects.

E25 | Bounded resource reuse: allocate a fixed number of photo textures and sprites when size or dimensions change.

Final/Assets/0_Scripts/Smartphone/CameraApp.cs
Original lines 1400-1426

```
1400        int size = Mathf.Max(1, maxStoredPhotos);
1401        if (photoTextures != null && photoTextures.Length == size && poolWidth == w && poolHeight == h
    > ) return;
1402
1403        if (photoTextures != null)
1404        {
1405            for (int i = 0; i < photoTextures.Length; i++) if (photoTextures[i] != null) Destroy(photo
    > Textures[i]);
1406            for (int i = 0; i < photoSprites.Length; i++) if (photoSprites[i] != null) Destroy(photoSp
    > rites[i]);
1407        }
1408
1409        photoTextures = new Texture2D[size];
1410        photoSprites = new Sprite[size];
1411        poolWidth = w;
1412        poolHeight = h;
1413        photoWriteIndex = 0;
1414
1415        Rect full = new Rect(0, 0, w, h);
1416        for (int i = 0; i < size; i++)
1417        {
1418            var tex = new Texture2D(w, h, TextureFormat.RGBA32, false, false)
1419            {
1420                name = "CameraApp_Photo_" + i,
1421                wrapMode = TextureWrapMode.Clamp,
1422            };
1423            photoTextures[i] = tex;
1424            // FullRect skips the alpha-driven mesh generation Sprite.Create does by default.
1425            photoSprites[i] = Sprite.Create(tex, full, new Vector2(0.5f, 0.5f), 100f, 0, SpriteMeshTyp
    > e.FullRect);
1426        }
```

The guard checks both capacity and dimensions, so a compatible existing buffer is retained. Texture2D[] and Sprite[] contain reusable object references. The for loop creates matching texture/sprite pairs once, and DoCaptureAndProcess selects photoTextures[photoWriteIndex] before advancing the index modulo the array length. Modulo wraps the write position to zero after the last slot. This is a bounded ring buffer, not the same borrowing API as ObjectPool. Its intended effect is to keep photo resource allocation steady after initialization, including when the player repeatedly takes pictures.

E26 | Cache lookup: accept an entry only when its full position key matches and its search depth is sufficient.

Final/Assets/0_Scripts/ChessMinigame/Engine/TranspositionTable.cs
Original lines 98-112

```
98              Entry entry = entries[key & indexMask];
99
100             if (entry.Key != key)
101             {
102                 move = EngineMove.None;
103                 staticEval = SearchEngine.NoEvalSentinel;
104
105                 return false;
106             }
107
108             move = entry.Move;
109             staticEval = entry.StaticEval;
110
111             if (entry.Depth < depth)
112                 return true;
```

The bitwise expression key & indexMask selects a table slot cheaply because its capacity is a power of two. The stored full key is then compared because different positions can map to the same slot. A matching entry with insufficient depth can still contribute a preferred move, but cannot supply a sufficiently deep score. Later code also checks whether the stored score is exact or only an upper/lower bound. SearchEngine consumes this information to avoid repeating valid work rather than treating every cache hit as an exact answer.

- SOLID connection: TranspositionTable owns replacement, key validation and score-bound storage separately from SearchEngine's traversal. CameraApp demonstrates useful resource discipline, although pooling alone is a performance technique, not proof of SRP or DIP.
- The phone's texture buffer is bounded, but GalleryApp.AddPhotoToGallery still instantiates a new UI item per shot. Existing gallery entries can reference a reused sprite whose pixels later change. Do not claim the entire gallery is a bounded immutable photo archive.
- AudioManager's coroutine depends on normal completion to return a source. Its current code does not express a comprehensive shutdown/release policy, so cancellation and object lifetime still deserve attention before describing the pool as fully robust.

The useful distinction

Pool: reuse an object after a borrower finishes. Ring buffer: overwrite a fixed sequence of storage slots. Cache: reuse a previous answer when its key and validity conditions match. Flyweight: share stable intrinsic data while keeping each instance's changing context separate. Naming the exact mechanism makes the design easier to explain and maintain.

11. Snapshots, undo records and cloning

Capture enough state for a specific purpose without pretending every snapshot is a complete Memento implementation.

There are three related state-management ideas in this project. SaveSystem captures a checkpoint DTO for disk persistence. Position keeps private undo records so chess search can reverse hypothetical moves. BoardState.Clone creates a separate position for background AI work. Each preserves state for a different lifetime and purpose. Calling all three Memento would hide useful distinctions. Checkpoint saving is Memento-like snapshot persistence; the private chess undo record is closer to encapsulated originator-owned restoration; cloning creates an independent working copy rather than a historical undo command.

E27 | Checkpoint snapshot: capture identity, scene, story resume point, progression, clock and run flags.

Final/Assets/0_Scripts/Persistence/SaveSystem.cs
Original lines 216-242

```
216         var data = new SaveData
217         {
218             checkpointId = checkpointId,
219             label = label,
220             savedAtUtc = DateTime.UtcNow.ToString("o"),
221             scenePath = SceneManager.GetActiveScene().path,
222             resumeNodeId = resumeNodeId,
223         };
224 
225         GameProgressManager progress = GameProgressManager.Instance;
226 
227         if (progress != null)
228         {
229             data.progressStage = (int)progress.CurrentStage;
230 
231             foreach (KeyValuePair<string, GameEventState> e in progress.GetEventSnapshot())
232                 data.events.Add(new SavedEvent(e.Key, e.Value));
233         }
234 
235         if (TimeManager.Instance != null)
236         {
237             data.hasTimeOfDay = true;
238             data.timeOfDayTicks = TimeManager.Instance.CurrentTime.Ticks;
239         }
240 
241         foreach (KeyValuePair<string, int> flag in flags)
242             data.flags.Add(new SavedFlag(flag.Key, flag.Value));
```

The object initializer new SaveData { ... } constructs a plain serializable data object. It contains stable scene and story identifiers rather than attempting to serialize every live MonoBehaviour. The foreach loop converts event snapshot entries into SavedEvent values because Unity's JsonUtility data layout uses fields and lists rather than dictionaries. Nullable dependency checks let optional progress and time systems be absent. Capturing explicit data creates a visible persistence contract: a future schema change is a data-format decision, not an accidental consequence of adding a runtime component field.

A concrete scenario is returning to a cabin checkpoint after leaving the player mid-dialogue. ContinueFromSave loads the saved scene and holds story startup. RestoreWorld restores progression, time, flags and player pose before allowing the runner to start at the saved node. NPC placements and props are deliberately reconstructed by that node's setup actions. SaveData therefore does not represent a complete frozen world. SaveSystem separately uses a temporary file, replacement and backup fallback to reduce the chance that an interrupted write destroys the previous checkpoint.

E28 | Undo restoration: reconstruct captured pieces and restore special-move rights, counters and the exact previous hash.

Final/Assets/0_Scripts/ChessMinigame/Engine/Position.cs
Original lines 309-325

```
309             if (flag == EngineMove.FlagEnPassant)
310             {
311                 int capturedSquare = us == ChessPieces.White ? to - 8 : to + 8;
312                 int captured = undo.CapturedPiece;
313 
314                 AddPiece(capturedSquare, ChessPieces.ColorOf(captured), ChessPieces.TypeOf(captured));
315             }
316             else if (undo.CapturedPiece != ChessPieces.NoPiece)
317             {
318                 int captured = undo.CapturedPiece;
319                 AddPiece(to, ChessPieces.ColorOf(captured), ChessPieces.TypeOf(captured));
320             }
321 
322             castlingRights = undo.Castling;
323             enPassantSquare = undo.EnPassant;
324             halfMoveClock = undo.HalfMove;
325             hashKey = undo.Key;
```

Position.MakeMove stores an UndoInfo before changing the board. The private struct contains the move, captured piece, castling rights, en passant square, half-move counter and previous key. UnmakeMove consumes that record and reverses the changed pieces. The excerpt's branch handles en passant's unusual captured square; the final assignments restore values that cannot reliably be inferred from piece placement alone. The private nested type keeps search clients from interpreting or modifying the saved record. Search callers ask Position to make and unmake moves instead of rebuilding its internals themselves.

E29 | Clone isolation: construct another BoardState and copy the engine position into it.

Final/Assets/0_Scripts/ChessMinigame/BoardState.cs
Original lines 105-112

```
105     public BoardState Clone()
106     {
107         BoardState clone = new BoardState();
108         clone.position.CopyFrom(position);
109         clone.InvalidateCache();
110 
111         return clone;
112     }
```

- ChessGameManager.RequestNpcMove supplies board.Clone() to NPCChessController. The worker can explore moves by mutating its own Position without modifying the board the player is viewing. Position.CopyFrom copies its arrays and relevant repetition history, not just another reference to the original object.
- SOLID connection: Position owns its representation and reversal rules, supporting encapsulation and SRP. SaveData separates the persisted format from the component doing file operations. Neither improvement automatically satisfies DIP: SaveSystem still reaches concrete global managers.
- A ChessMove is a value describing a move, and the visible move log stores notation strings. Those facts do not establish the Command pattern: there is no common command object encapsulating execute and undo behavior for the application.

Scope and limits

The checkpoint DTO is public and SaveSystem knows several systems' data, so this is not a textbook opaque GoF Memento with a caretaker unable to inspect the state. BoardState.Clone is useful copying, but the project does not expose a general polymorphic Prototype family. The appropriate claim is purpose-specific snapshots and clone isolation.

12. Data-driven configuration and component composition

Move tunable values into data and add behavior through stable extension points.

The project regularly separates authored choices from execution code. Examples include ChessDifficultyProfile, TimeSettings, SignalWordTeamBoardData, LiquidFeelSettings and the phone feel configuration. ScriptableObject makes suitable data available as an Inspector-editable Unity asset; Serializable classes can instead embed settings within another component. These are Unity data-driven architecture practices, not automatically GoF Strategy or Flyweight. Their value is concrete: a designer can adjust timing, clue content, difficulty or visual response while the consuming algorithm stays the same.

E30 | Boundary conversion: an authored Unity asset becomes plain engine strength settings.

`Final/Assets/0_Scripts/ChessMinigame/ChessDifficultyProfile.cs`
`Original lines 57-79`

```
57      public EngineStrengthSettings ToEngineSettings()
58      {
59          int thinkTime = ThinkTimeMilliseconds;
60          int depth = MaxSearchDepth;
61
62          // Assets authored before the rewrite only carry a depth; honour it rather than
63          // silently playing at a completely different strength.
64          if (thinkTime <= 0)
65          {
66              thinkTime = 0;
67              depth = Mathf.Clamp(SearchDepth, 1, 64);
68          }
69
70          return new EngineStrengthSettings
71          {
72              MaxDepth = depth,
73              MaxTimeMilliseconds = thinkTime,
74              MaxNodes = MaxNodes,
75              SelectionMarginCentipawns = MoveChoiceMarginCentipawns,
76              SlipChancePerMille = Mathf.RoundToInt(Mathf.Clamp(SlipChancePercent, 0f, 100f) * 10f),
77              SlipMarginCentipawns = Mathf.Max(SlipMarginCentipawns, MoveChoiceMarginCentipawns),
78              TranspositionMegabytes = TranspositionMegabytes
79          };
```

ToEngineSettings reads the profile and returns a new EngineStrengthSettings value. The object initializer maps authoring names to runtime engine fields. Mathf.Clamp and Mathf.Max normalize values at the boundary, and the legacy branch preserves older SearchDepth assets when a time budget was not supplied. This lets ChessBrain consume plain settings without depending on the ScriptableObject type. For example, a beginner profile can limit depth and nodes and allow imperfect root-move choices, while a stronger profile uses the same search implementation with a larger budget.

E31 | Open/Closed extension point: dispatch the focused click through BaseApp without checking concrete app types.

Final/Assets/0_Scripts/Smartphone/Smartphone.cs
Original lines 386-399

```
386     private void RunFocusAction()
387     {
388         if (IsPauseHeld || !feel.focusHidesPhoneCursor)
389             return;
390 
391         apps ??= GetComponentsInChildren<BaseApp>(true);
392 
393         foreach (BaseApp app in apps)
394         {
395             // An app is open exactly while its page is active.
396             if (app != null && app.isActiveAndEnabled && app.FocusAction())
397                 return;
398         }
399     }
```

The null-coalescing assignment ??= discovers the child BaseApp components once when the cache is absent. The foreach variable has the abstract base type, so the caller uses the common FocusAction contract. Because that method is virtual, runtime dispatch selects CameraApp's shutter behavior, LightApp's flashlight toggle or MusicApp's play/pause behavior. Returning true means the active app handled the action and stops the loop; BaseApp's default false is a valid no-action result. A new app can override this operation without adding another type check to Smartphone.

E32 | Composition: the cursor coordinator creates separate visual and selection-affinity collaborators.

Final/Assets/0_Scripts/Smartphone/WorldCanvasVirtualCursor.cs
Original lines 175-187

```
175         if (useCircleVisual && cursorRect != null && canvasRect != null)
176         {
177             visual = new PhoneCursorVisual(
178                 cursorRect,
179                 canvasRect.rect.width * circleDiameter,
180                 circleColor,
181                 outlineColor,
182                 hoverGrow,
183                 pressSqueeze,
184                 hoverFillOpacity,
185                 withPupil: true);
186             affinity = new SelectionAffinity(gameObject.scene, EventCamera);
187         }
```

The new expressions construct ordinary helper objects rather than adding more responsibilities through inheritance. PhoneCursorVisual owns the eye, ripple and other drawn feedback; SelectionAffinity finds an eligible nearby target and coordinates its fill. WorldCanvasVirtualCursor drives pointer routing and calls those collaborators. LiquidSelectionFill in turn composes text deformation and shape helpers, while LiquidBalloonShape centralizes deformation geometry. This arrangement lets a visual change remain in presentation helpers instead of being intertwined with press, release, drag and scrolling logic.

- SRP: settings describe choices; behavior consumes them; visual helpers maintain their own state. Signal Words similarly creates fresh runtime cards from authored board data instead of marking the ScriptableObject's cards revealed.
- OCP: FocusAction is a strong behavioral extension point. Data changes also extend content without source edits. A new camera mode or a new built-in difficulty enum still requires edits to current switches and blending logic, so OCP is selective.
- LSP: an app that has no focus action can return false without violating the documented contract. This is better evidence than merely observing an inheritance relationship. It does not establish that all BaseApp initialization and lifecycle behavior is consistent.

- DIP: plain engine settings remove a Unity asset dependency from the engine, but the constructor calls shown here still use concrete helpers. Smartphone supports injected IPlayerInput while also retaining an InputManager singleton fallback; the larger system is not fully inverted.

> **Practical limitations**
>
> Configuration does not make a component single-purpose by itself. CameraApp still coordinates many rendering and UI responsibilities. Several phone classes duplicate spring integration code. Shared settings are mutable, so a runtime edit may affect multiple consumers. BaseApp's large presentation surface and derived Unity lifecycle methods also need consistent initialization rules.

13. Flyweight-style tree geometry and spatial partitioning

Share the expensive description of a tree, keep each tree's pose separate, and search only nearby cells.

TreeLeap supplies a stronger Flyweight-style example than a generic shared settings asset. TrunkProfile stores expensive baked bark data: vertices, triangles, polar radii, clearance and height buckets. Many TrunkShape objects reference one profile while keeping their own transforms, scales, world position and sway state. The shared geometry is intrinsic state; each placed tree's context is extrinsic state. TrunkLibrary resolves a profile by prefab or mesh. This arrangement avoids copying the same bark arrays for every world instance of an identical tree.

E33 | Flyweight-style split: retain the shared profile reference while computing per-instance world-space context.

`Final/Assets/_TreeLeap/Scripts/TrunkShape.cs`
`Original lines 121-133`

```
121         public TrunkShape(int id, TrunkProfile p, Matrix4x4 localToWorld)
122         {
123             this.id = id;
124             profile = p;
125             l2w = localToWorld;
126             w2l = localToWorld.inverse;
127             normalMatrix = w2l.transpose;
128             basePosition = l2w.MultiplyPoint3x4(Vector3.zero);
129             swayPivot = basePosition;
130             widthScale = l2w.MultiplyVector(Vector3.right).magnitude;
131             heightScale = l2w.MultiplyVector(Vector3.up).magnitude;
132             label = $"{p.name}#{id}";
133         }
```

The assignment profile = p retains a reference; it does not clone the profile arrays. In contrast, localToWorld is stored for this instance, its inverse becomes w2l, and its transpose supplies a normal transform. The width and height scales are derived from transformed basis vectors. Surface queries can therefore evaluate shared local bark geometry and transform the result into the particular tree's world space. Two identical tree prefabs can have different sizes, rotations and locations while sharing the same baked description.

E34 | Many terrain instances reuse the resolved prototype profile; overlapping duplicates are merged.

Final/Assets/_TreeLeap/Scripts/TreeRegistry.cs
Original lines 38-56

```
                for (int idx = 0; idx < instances.Length; idx++)
                {
                    var ti = instances[idx];
                    if (ti.prototypeIndex < 0 || ti.prototypeIndex >= profiles.Length) continue;
                    var prof = profiles[ti.prototypeIndex];
                    if (prof == null) continue;
                    Vector3 wp = tp + Vector3.Scale(ti.position, size);
                    var key = (protos[ti.prototypeIndex].prefab ? protos[ti.prototypeIndex].prefab.Get
InstanceID() : 0,
                        (long)Mathf.Round(wp.x * 20f), (long)Mathf.Round(wp.z * 20f));
                    if (seen.TryGetValue(key, out var existing))
                    {
                        // the same tree drawn by an overlapping terrain: one trunk, remember every in
stance
                        existing.terrainRefs.Add((terrain, idx));
                        reg.duplicatesMerged++;
                        continue;
                    }
                    var m = Matrix4x4.TRS(wp, Quaternion.Euler(0f, ti.rotation * Mathf.Rad2Deg, 0f),
                        new Vector3(ti.widthScale, ti.heightScale, ti.widthScale));
                    var shape = new TrunkShape(reg.trunks.Count, prof, m) { terrain = terrain };
```

Before this loop, Build resolves one profile for each terrain tree prototype. The prototypeIndex selects the same TrunkProfile for every instance using that prototype. Matrix4x4.TRS supplies each instance's position, rotation and scale, and new TrunkShape combines that context with the shared profile. The seen dictionary also merges matching overlapping terrain instances while recording their terrain references. This is an additional deduplication rule, not the Flyweight definition itself. Derived cylinder fallbacks are cached by prefab, so those profiles are also reused during registry construction.

E35 | Spatial Partition: query an XZ grid, then use exact horizontal distance to reject candidates outside the radius.

Final/Assets/_TreeLeap/Scripts/TreeRegistry.cs
Original lines 126-142

```
        public void Query(Vector3 p, float radius, List<TrunkShape> results)
        {
            results.Clear();
            int x0 = Mathf.FloorToInt((p.x - radius) / Cell), x1 = Mathf.FloorToInt((p.x + radius) / C
ell);
            int z0 = Mathf.FloorToInt((p.z - radius) / Cell), z1 = Mathf.FloorToInt((p.z + radius) / C
ell);
            float r2 = radius * radius;
            for (int x = x0; x <= x1; x++)
                for (int z = z0; z <= z1; z++)
                {
                    if (!_grid.TryGetValue(Key(x, z), out var list)) continue;
                    foreach (var s in list)
                    {
                        float dx = s.basePosition.x - p.x, dz = s.basePosition.z - p.z;
                        if (dx * dx + dz * dz <= r2) results.Add(s);
                    }
                }
        }
```

TreeRegistry.Add places each trunk in an 8-metre XZ cell. Query computes the minimum and maximum cell coordinates intersecting the requested radius, then visits only those buckets. The expression dx * dx + dz * dz <= r2 compares squared distances and avoids a square root for every candidate. results.Clear() followed by results.Add(...) reuses the caller's list rather than returning a newly allocated collection. The long key combines integer X and Z coordinates. This is a spatial grid or spatial hash-style partition, not merely a database named Registry.

- Actual flow: TreeLeapCreature builds the registry once during setup. Ground movement requests nearby trunks for avoidance, and jump selection/path checks query around the relevant origin or segment. A creature in one corner of the forest need not scan every distant tree for each local decision.
- Example: 200 world instances of one tree can share one profile's baked triangles while maintaining 200 poses. A local jump query filters those instances by nearby cells and radius before more detailed bark or path checks.
- SOLID connection: the library owns shared descriptions and lookup, TrunkShape owns instance geometry queries, and TreeRegistry owns collection and spatial search. These are useful SRP boundaries and composition. No interface abstraction here establishes DIP or LSP by itself.

Assumptions and limits

Flyweight-style sharing relies on treating profile geometry as stable after baking; the public fields do not enforce immutability. Changing a shared profile affects all referencing trunks. The grid indexes rest/base positions and has no general moving-object reindex operation. Large radii or crowded cells still examine many candidates. TrunkLibrary's dictionaries accelerate lookup, but those dictionaries alone are caches, not the Flyweight pattern.

14. S - Single Responsibility Principle

Keep each responsibility with the class that has the clearest reason to change.

Single Responsibility means a class should have one coherent reason to change. It does not mean one method per class, and a long implementation is not automatically wrong. In this project, the boundary system provides a clear example: spatial rules, selection of the active boundary, and visual presentation are separate responsibilities. A change to how the zone calculates a permitted position can stay separate from a change to the shader effect.

E36 | The zone owns the geometric rule for clamping a position.

`Final/Assets/0_Scripts/Player/PlayerBoundary/PlayerBoundaryZone.cs`
`Original lines 74-87`

```
74      public Vector3 ClampPosition(Vector3 worldPosition)
75      {
76          if (boxCollider == null)
77              return worldPosition;
78
79          Vector3 local = transform.InverseTransformPoint(worldPosition);
80          Vector3 halfSize = boxCollider.size * 0.5f;
81          Vector3 center = boxCollider.center;
82
83          local.x = Mathf.Clamp(local.x, center.x - halfSize.x, center.x + halfSize.x);
84          local.z = Mathf.Clamp(local.z, center.z - halfSize.z, center.z + halfSize.z);
85
86          return transform.TransformPoint(local);
87      }
```

The method accepts and returns a `Vector3`, so its job is visible in its signature. `InverseTransformPoint(worldPosition)` converts the input into the zone's coordinate system. The two `Mathf.Clamp` calls constrain local X and Z, and `TransformPoint(local)` returns a world-space result. The null guard returns the original position when no collider exists. There is no material update, sound playback, or decision about which story boundary should be active in this calculation.

E37 | The manager delegates the calculation to the selected active zone.

`Final/Assets/0_Scripts/Player/PlayerBoundary/PlayerBoundaryManager.cs`
`Original lines 73-81`

```
73      public Vector3 ClampPosition(Vector3 desiredPosition)
74      {
75          PlayerBoundaryZone zone = ActiveZone;
76
77          if (zone == null)
78              return desiredPosition;
79
80          return zone.ClampPosition(desiredPosition);
81      }
```

`PlayerBoundaryZone zone = ActiveZone` resolves the selected zone through the manager's existing property. If none is usable, returning `desiredPosition` preserves unrestricted movement. Otherwise `zone.ClampPosition(desiredPosition)` delegates the calculation. PlayerMovement calls this manager while applying movement. The manager centralizes which boundary applies, rather than copying each zone's coordinate mathematics into the movement controller. This also avoids giving every zone responsibility for discovering or moving the player.

E38 | The visual driver translates an intensity value into rendering state.

`Final/Assets/0_Scripts/Player/PlayerBoundary/BoundaryDistortionDriver.cs`
`Original lines 143-152`

```
143     private void Apply(float value)
144     {
145         intensity = value;
146
147         if (distortionMaterial != null)
148             distortionMaterial.SetFloat(IntensityID, value);
149
150         if (boundaryVolume != null)
151             boundaryVolume.weight = Mathf.Clamp01(value * volumeWeightMultiplier);
152     }
```

Here `private void Apply(float value)` has a different responsibility: presentation. It stores the current intensity, writes the shader parameter through `SetFloat`, and sets the optional volume's weight. `Mathf.Clamp01` keeps that weight in its valid range. The driver's update obtains boundary proximity from the manager and applies its response curve. Consequently, tuning the visual ramp does not require changing the zone's clamping rule. This is separation of concerns expressed through actual calls between components.

The player state system shows the same intent: BasePlayerState describes a profile, PlayerStateStack owns suspension and restoration, and PlayerStateController applies movement, camera, and blocker effects. The Observer examples also help: movement announces footsteps while PlayerFootstepController decides their sounds. Each division keeps a different kind of change in a more focused place.

Where responsibility is still mixed

PlayerInventory handles inventory and equipment data alongside input polling, UI text, overlay visibility, and audio. CameraControllerNew combines look conversion, anchor filtering, bob, stairs, zoom, lens control, and focus. GrabbableItem spans holding physics, input, prompts, inventory, and drop-zone lookup. These are meaningful areas for gradual extraction; file length alone is not the reason.

A focused improvement would preserve existing public entry points while moving inventory data and equipment operations into a model, then letting a UI component observe changes. Assessment: strong local examples of Single Responsibility, with partial adherence in several broad gameplay controllers.

15. O - Open/Closed Principle

Add a new behavior through an existing extension point without rewriting its consumers.

Open/Closed means software should support a useful kind of extension while keeping established consumers stable. It does not promise that no file will ever change. The important question is which variation the design anticipates. This project anticipates different interactable objects, different placement targets, and different sources of car input. Interfaces give those variations a shared contract, and their consumers actually use that contract.

E39 | PlayerUI discovers an interaction capability and calls it polymorphically.

`Final/Assets/0_Scripts/PlayerUI/PlayerUI.cs`
`Original lines 147-172`

```
147                 if (h.collider.TryGetComponent<IInteractable>(out IInteractable i))
148                 {
149                     if (i.CanInteract)
150                     {
151                         float DistanceFromPlayer = Vector3.SqrMagnitude(playerCamera.transform.positio
  >  n - h.collider.transform.position);
152                         if (i.MinInteractionDistance * i.MinInteractionDistance > DistanceFromPlayer)
153                         {
154                             CurrentInteractable = null;
155                             // i.ToggleVisualOutline(false);
156                             // RETURN SINCE ITS TOO CLOSE
157                             return false;
158                         }
159                         //if (CurrentInteractable != null)
160                         //{
161                         //    if (CurrentInteractable != i)
162                         //    {
163                         //        CurrentInteractable.ToggleVisualOutline(false);
164                         //    }
165                         //}
166                         CurrentInteractable = i;
167                         // i.ToggleVisualOutline(true);
168                         if (InteractBuffer.hasBuffer)
169                         {
170                             InteractBuffer.Consume();
171                             return i.Interact();
172                         }
```

`TryGetComponent<IInteractable>` is a generic component query for the interface, rather than a query for Door, Chair, or PlateSpot. The `out IInteractable i` declaration creates a variable typed as that contract. PlayerUI checks shared properties, records the target, consumes its buffered interaction, and calls `i.Interact()`. The concrete object's implementation runs through that interface call. A new object implementing the same contract can enter this flow without another type branch in PlayerUI, provided its collider, layer, and other setup satisfy the existing interaction rules.

E40 | A placement hint asks the target what it accepts instead of recognizing target classes.

Final/Assets/0_Scripts/Interaction/PlacementHint.cs
Original lines 132-142

```
132     private void LateUpdate()
133     {
134         GrabbableItem held = GrabbableItem.Held;
135         bool show = held != null && target.WouldAccept(held) && IsInRange();
136 
137         if (show && held != ghostSource)
138             BuildGhost(held);
139 
140         bool targeted = show && target.IsTargetedBy(held);
141 
142         float dt = Time.deltaTime;
```

PlacementHint stores its target as `IPlacementTarget`, resolved in Awake using `GetComponent<IPlacementTarget>()`. In this excerpt, `target.WouldAccept(held)` decides whether a ghost should appear, and `target.IsTargetedBy(held)` decides whether it is actively aimed at. PlateSpot and GrabbableDropZone implement these questions using different rules. The hint never needs an `if` identifying either concrete target. Adding another target can reuse the visual feedback component by implementing those same questions and exposing a placement anchor.

The car provides another working extension point: SUVCarController accepts `ICarInput` through SetInputSource and reads the five controls from that abstraction. SUVCarInput supplies player controls; IntroCruiseInput supplies the scripted intro and later restores the default source. This is the Strategy example from the pattern discussion, and it also demonstrates Open/Closed: the physics controller need not know how each provider decides its values.

E41 | Camera behavior is separated into states, but selecting them remains explicitly wired.

Final/Assets/0_Scripts/Player/StateMachineCamera/PlayerCameraStates.cs
Original lines 91-117

```
91      private BaseCameraState ResolveState(CameraStateChangeEvent e)
92      {
93          switch (e.NewCameraMode)
94          {
95              case CameraMode.FreeLook:
96                  return freeLookState;
97 
98              case CameraMode.PhoneLook:
99                  return phoneLookState;
100 
101             case CameraMode.Locked:
102                 return lockedState;
103 
104             case CameraMode.HeadFollow:
105                 return headFollowState;
106 
107             case CameraMode.Sitting:
108                 sittingState.SetReference(e.SitTransform);
109                 return sittingState;
110 
111             case CameraMode.Dialogue:
112                 dialogueState.SetParameters(e.TargetNpcName, e.PanDuration);
113                 return dialogueState;
114 
115             case CameraMode.BoardGameTopDown:
116                 boardGameState.SetBoardContext(e.BoardGameCamSettings);
117                 return boardGameState;
```

This switch marks a deliberate limit. `switch (e.NewCameraMode)` maps a closed enum to preconstructed fields. Parameter-bearing cases also call concrete setters such as `SetReference` or `SetParameters`. Adding a new camera mode therefore requires edits to the enum, construction, and this selection code. Polymorphic Update calls isolate the

behavior, but they do not make registration itself open to extension. The player state controller likewise constructs a fixed collection of states.

> **Focused improvement**
>
> Keep the explicit switch when the supported modes are intentionally fixed. If camera modes become independently contributed features, introduce a registration map or factory boundary and a documented parameter contract. Do not add indirection solely to remove a small, stable switch.

Assessment: strong Open/Closed support at interaction, placement, and vehicle-input boundaries; partial support for adding new player or camera state types because their registration remains centralized and concrete.

16. L - Liskov Substitution Principle

Shared types must promise consistent behavior, not merely matching method names.

Liskov Substitution means code using a base type or interface should continue to work correctly when given a valid implementation of that type. Matching the compiler's required members is only the starting point. Callers also rely on meaning: what a return value says, whether a property setter changes its value, which inputs are accepted, and which effects occur. The project contains useful structural support for substitution, plus a specific interaction-contract inconsistency.

E42 | The player-state abstraction explicitly makes validation part of its contract.

`Final/Assets/0_Scripts/Player/StateMachinePlayer/BasePlayerState.cs`
`Original lines 23-41`

```
23      public abstract PlayerStateName Name { get; }
24
25      /// <summary>
26      /// Checked BEFORE the stack commits to this state, and again before a suspended frame is
27      /// restored. Returning false leaves the player where they are instead of stranding them
28      /// in a half-entered state - which is what happened when states validated inside OnEnter
29      /// and bailed out after the machine had already switched.
30      /// </summary>
31      public virtual bool Validate(object payload, out string reason)
32      {
33          reason = null;
34          return true;
35      }
36
37      /// <summary>
38      /// What movement mode, camera and blockers this state needs. Called on entry, on resume
39      /// and whenever the payload is refreshed, so it must be a pure function of the payload.
40      /// </summary>
41      public abstract PlayerStateProfile BuildProfile(object payload);
```

`abstract` requires concrete states to supply their identity and BuildProfile result. `virtual` supplies a default Validate implementation while allowing a state to explain why its payload is unusable. The `out string reason` parameter returns that explanation alongside the Boolean result. The comment on BuildProfile also documents an important behavioral expectation: it should be a pure function of its payload. This is stronger guidance than an undocumented collection of override methods.

E43 | The consumer honors the validation contract before suspending the current state.

Final/Assets/0_Scripts/Player/StateMachinePlayer/PlayerStateStack.cs
Original lines 87-106

```
87      public bool Push(BasePlayerState state, object payload = null)
88      {
89          if (state == null)
90          {
91              Debug.LogError($"{nameof(PlayerStateStack)}.{nameof(Push)} called with no state.");
92              return false;
93          }
94
95          if (!state.Validate(payload, out string reason))
96          {
97              Debug.LogError($"[PlayerStates] Refused to enter {state.Name}: {reason}. Staying in {Curre
  > ntName}.");
98              return false;
99          }
100
101         // Already on top. Not an interruption, so it must not push a second frame - two
102         // pushes of the same halt would otherwise need two pops to undo. Refresh in place.
103         if (ReferenceEquals(Current, state))
104         {
105             RefreshTop(payload);
106             return true;
```

Push accepts a `BasePlayerState`, so it handles concrete states through the same base abstraction. `!state.Validate(...)` rejects an invalid request and returns before the later suspension and entry logic. DialogueState validates its NPC payload; SittingState validates its seat reference. Rejecting such inputs is part of the framework's declared protocol, rather than an unexpected exception introduced by one subtype. On restoration, the stack validates suspended payloads again and discards stale frames. Those choices support consistent lifecycle behavior when the world has changed.

The evidence here is structural and contractual. It does not prove every state preserves all invariants under reentrant calls, destroyed objects, or arbitrary payloads. Behavioral tests would be needed for that stronger claim. Likewise, the fact that the car uses both player and scripted ICarInput providers demonstrates a real substitution seam, but a provider must still honor the documented zero-to-one control ranges.

E44 | PlateSpot satisfies the interface syntax while silently ignoring writes to InteractText.

Final/Assets/0_Scripts/Interaction/PlateSpot.cs
Original lines 81-97

```
81      public bool CanInteract
82      {
83          get => interactionEnabled && Time.time >= nextInteractTime && (CanPlace || CanTakeBack);
84          set => interactionEnabled = value;
85      }
86
87      public string InteractText
88      {
89          get => CanPlace ? placeText : takeBackText;
90          set { /* Driven by state. Required by IInteractable. */ }
91      }
92
93      public float MinInteractionDistance
94      {
95          get => minInteractDistance;
96          set => minInteractDistance = value;
97      }
```

IInteractable declares `InteractText { get; set; }`. Most implementations store an assigned value, but PlateSpot's getter computes a prompt from CanPlace and its setter does nothing. PlateStack uses the same approach. The empty

`set { ... }` compiles, yet a client assigning a common interaction prompt cannot expect the assignment to work uniformly. This is a concrete contract mismatch and a potential Liskov violation if callers rely on ordinary setter semantics. The interface does not currently document a narrower promise.

> **Repair the promise**
>
> Make the common prompt property read-only when consumers only display it, and provide a separate explicit configuration API where mutation is supported. Also define Interact's Boolean meaning consistently: Door returns false when blocked, while several older interactables return true even when they perform no action.

Assessment: good substitution structure in states and car inputs, with an unresolved interaction contract. Inheritance or interface use alone should not be presented as proof of Liskov compliance.

17. I - Interface Segregation Principle

Give a client the capabilities it needs without making it depend on unrelated features.

Interface Segregation asks whether an interface is shaped around its clients' needs. A focused interface lets consumers and alternate implementations depend on a small, relevant capability. The goal is not the largest possible number of tiny interfaces. The project has strong examples where a narrow contract is consumed in practice, and weaker examples where a broad shared contract or unused declaration does less architectural work.

E45 | The car asks only for its five driving controls.

`Final/Assets/0_Scripts/Vehicle/ICarInput.cs`
`Original lines 6-13`

```
6   public interface ICarInput
7   {
8       float Accelerate { get; }
9       float SteerLeft { get; }
10      float SteerRight { get; }
11      float Reverse { get; }
12      float Brake { get; }
13  }
```

Each `float ... { get; }` is a read-only query from the consumer's perspective. SUVCarController can read accelerator, reverse, braking, and steering without gaining access to smartphone buttons, inventory navigation, hardware devices, or Unity action-map control. SUVCarInput and IntroCruiseInput can satisfy this focused contract using different sources. The interface is therefore both small and operationally relevant: actual vehicle code reads it and actual providers implement it.

E46 | Placement feedback has a focused contract separate from activating an interaction.

Final/Assets/0_Scripts/Interaction/IPlacementTarget.cs
Original lines 8-21

```
8   public interface IPlacementTarget
9   {
10      /// <summary>Where the item ends up when it is placed here.</summary>
11      Transform PlacementAnchor { get; }
12
13      /// <summary>Whether this target would take <paramref name="item"/> right now.</summary>
14      bool WouldAccept(GrabbableItem item);
15
16      /// <summary>
17      /// Whether the player is aiming at this target with <paramref name="item"/> - that is,
18      /// whether pressing [E] right now would put it here.
19      /// </summary>
20      bool IsTargetedBy(GrabbableItem item);
21  }
```

`Transform PlacementAnchor { get; }` supplies a pose reference without a setter that would let a visual client rewrite placement configuration. `WouldAccept` and `IsTargetedBy` answer the two questions PlacementHint needs. The interface does not require placement execution, audio playback, inventory editing, or outline control. PlateSpot implements both IInteractable and IPlacementTarget, while GrabbableDropZone supplies placement queries for its own trigger-based workflow. Composition lets one object expose several capabilities while each consumer selects the relevant one.

IPlayerStateEffects is another focused boundary: the stack asks only to Apply or Revert a PlayerStateProfile. It does not need the rest of PlayerStateController's dialogue, seating, camera-override, or movement APIs. The Dependency Inversion discussion explains how that interface is injected. Its small surface is also an Interface Segregation benefit because the stack's dependency reflects its actual job.

E47 | One portion of IPlayerInput already spans inventory, phone controls, and cursor confirmation.

Final/Assets/0_Scripts/PlayerInput/IPlayerInput.cs
Original lines 77-102

```
77      // ---- Inventory ------------------------------------------------------------
78
79      InputButton Inventory { get; }
80      InputButton NextItem { get; }
81      InputButton PreviousItem { get; }
82      InputButton Equip { get; }
83
84      // ---- Phone, cursor and UI -------------------------------------------------
85
86      /// <summary>Tab. Raises or puts away the phone.</summary>
87      InputButton Smartphone { get; }
88
89      /// <summary>
90      /// X. With the phone up, swaps between the hand cursor that picks the phone up and the
91      /// phone's own screen cursor. (Named for what it used to do: move the phone directly.)
92      /// </summary>
93      InputButton DragPhone { get; }
94
95      /// <summary>Pointer held down - the "grab and drag" button. Was MousePressed.</summary>
96      InputButton PointerHold { get; }
97
98      /// <summary>
99      /// Confirm, pointer included: mouse left OR keyboard submit. For cursor-driven UI, where a
100     /// click is the obvious way to say yes.
101     /// </summary>
102     InputButton Submit { get; }
```

The complete IPlayerInput contract additionally includes movement, looking, locomotion buttons, interaction, and other UI controls. Centralizing device access is useful, but a camera consumer needs only a fraction of this surface. Any

alternate implementation or test fake must still supply the unrelated properties. That makes segregation coarse-grained. It is a tradeoff to assess from client needs rather than an automatic failure simply because the interface has many members.

> **A practical next step**
>
> If alternate input providers or focused tests become common, expose client-oriented views such as look input, locomotion input, and UI input, while allowing InputManager to implement them all. Avoid splitting every button into its own interface. Revisit IInteractable's writable prompt and outline obligations, whose mismatch is visible in the Liskov example.

IRotatable is a narrow interface, but searches found implementations without a consumer calling it polymorphically; it is only potential evidence. ICookable.cs is actually an empty MonoBehaviour class, not an interface. Assessment: strong segregation in car controls, placement queries, and state effects; partial segregation in the broad player-input and interaction contracts.

18. D - Dependency Inversion Principle

Keep stable policy dependent on contracts, and connect concrete implementations at a boundary.

Dependency Inversion means higher-level policy and lower-level details meet through abstractions. Dependency injection is one way to connect those abstractions to implementations; merely passing a concrete component into another class does not establish the principle. This project's strongest example is the player state stack. Its policy is how interruptions enter, suspend, resume, and exit. It does not need to know how a camera changes mode or how an interaction blocker is stored.

E48 | The state container receives its effects boundary through its constructor.

`Final/Assets/0_Scripts/Player/StateMachinePlayer/PlayerStateStack.cs`
`Original lines 36-50`

```
36  public sealed class PlayerStateStack
37  {
38      /// <summary>
39      /// Interruptions above the base state. Something pushing without ever popping should be
40      /// noisy and bounded rather than growing forever.
41      /// </summary>
42      private const int MaxDepth = 8;
43
44      private readonly List<Frame> frames = new List<Frame>();
45      private readonly IPlayerStateEffects effects;
46
47      public PlayerStateStack(IPlayerStateEffects effects)
48      {
49          this.effects = effects;
50      }
```

`private readonly IPlayerStateEffects effects` stores a contract rather than PlayerStateController itself. `readonly` prevents reassignment after construction; it does not claim the supplied implementation is immutable. The constructor parameter gives the caller responsibility for supplying that collaborator. Later stack operations invoke `effects.Apply(profile)` and `effects.Revert(profile)`, while state objects supply declarative PlayerStateProfile values. A focused substitute could record those calls without needing to implement a real camera. The stack still uses Unity logging, so this is not a claim that it is completely engine-independent.

E49 | The concrete controller translates the contract into movement, camera, and blocker operations.

Final/Assets/0_Scripts/Player/StateMachinePlayer/PlayerStateController.cs
Original lines 129-146

```
129     void IPlayerStateEffects.Apply(in PlayerStateProfile profile)
130     {
131         player.controller.SetMovementMode(profile.Movement);
132
133         player.CameraStates.SwitchCameraState(hasCameraOverride ? cameraOverride : profile.Camera);
134
135         if (profile.BlocksInteraction && PlayerGameState.Instance != null)
136             PlayerGameState.Instance.AddInteractionBlocker(profile.InteractionBlocker);
137     }
138
139     void IPlayerStateEffects.Revert(in PlayerStateProfile profile)
140     {
141         // Movement mode and camera mode are not restored here on purpose: whatever becomes
142         // current next applies its own, and there is always something becoming current.
143         // Only what was TAKEN has to be given back.
144         if (profile.BlocksInteraction && PlayerGameState.Instance != null)
145             PlayerGameState.Instance.RemoveInteractionBlocker(profile.InteractionBlocker);
146     }
```

The qualified syntax `void IPlayerStateEffects.Apply(...)` is an explicit interface implementation. It is reached through the interface, rather than becoming another ordinary public controller method. `in PlayerStateProfile profile` passes the struct by readonly reference. Apply changes the movement mode, chooses the state camera or override, and conditionally takes an interaction blocker. Revert releases the blocker it previously owned. This keeps world-facing details on the implementation side of the boundary while the stack owns sequencing.

E50 | The installer actually supplies IPlayerInput to compatible components in its hierarchy.

Final/Assets/0_Scripts/Player/PlayerControls/PlayerInstaller.cs
Original lines 29-38

```
29      var consumers = GetComponentsInChildren<MonoBehaviour>(true);
30
31      foreach (var component in consumers)
32      {
33        if (component is IInitializable<IPlayerInput> consumer)
34        {
35          consumer.Initialize(input);
36        }
37      }
38    }
```

Earlier in the method, the installer resolves InputManager.Instance into an `IPlayerInput` variable. `GetComponentsInChildren<MonoBehaviour>(true)` includes inactive children. The `is IInitializable<IPlayerInput> consumer` pattern both checks the generic initialization capability and binds a typed variable; `consumer.Initialize(input)` performs the injection. PlayerMovement, CameraControllerNew, and PlayerUI then read their injected interface. The concrete singleton lookup is concentrated at this wiring point, although those consumers also retain singleton fallbacks.

Vehicle input and placement feedback reinforce the same direction: SUVCarController accepts ICarInput through SetInputSource, and PlacementHint asks IPlacementTarget for acceptance and targeting. These are demonstrated dependencies on contracts. In contrast, direct calls to Player.Instance, PlayerGameState.Instance, NpcDatabase.Instance, and other concrete managers remain common. They make collaborators implicit and complicate isolated substitution, even where the surrounding code is otherwise well structured.

Keep the claim evidence-based

IAudioManager declares a contract but has no implementation or consuming call site in the reviewed project, so it does not demonstrate Dependency Inversion. A focused improvement is to inject the collaborators needed by a system at construction or initialization, and retain scene discovery at a small composition boundary where practical.

Assessment: strong local Dependency Inversion in the state-effects, car-input, and placement seams; partial project-wide adherence because global concrete services and fallback lookups remain part of normal execution.

19. Subsystem findings beyond the main patterns

Ordinary components, ownership safeguards, and unfinished abstractions also matter.

The worked examples concentrate on reusable collaborations. The remaining scripts include focused components, configuration, editor automation, procedural effects, and scaffolds. These can be evaluated for cohesion, ownership, encapsulation, and duplication without inventing a GoF label for each file.

Ambience, night vision, and intoxication

CrowFlock coordinates group decisions while CrowAgent owns individual activity, movement, animation overlays, and attention. CrowBaker performs branch and feeding-spot analysis in the editor, so mesh analysis does not become a recurring runtime cost. CrowFlockSettings separates tuning from execution. The split is useful, but the two runtime classes still have several responsibilities and depend concretely on each other. Their enum switches describe FSMs rather than interchangeable IState objects.

MothSwarmController reuses a particle buffer, validates settings, and interacts with a Light rather than requiring a specific phone app. That makes the swarm useful for different light sources. MothZoneTrigger separates zone entry/exit from swarm simulation. Its occupant set avoids releasing a zone when only one of a player's colliders leaves.

E51 | Zone exit and disable cleanup release only this zone's suppression

Final/Assets/0_Scripts/Ambience/Moths/MothZoneTrigger.cs
Original lines 104-129

```
104         }
105
106         OnZoneEntered?.Invoke();
107     }
108
109     private void OnTriggerExit(Collider other)
110     {
111         if (!occupants.Remove(other) || occupants.Count > 0)
112             return;
113
114         if (mode == ZoneMode.SuppressWhileInside)
115             ForEachTarget(swarm => swarm.RemoveSuppressor(this));
116
117         OnZoneExited?.Invoke();
118     }
119
120     // A zone that is switched off or destroyed while the player is standing in it must not
121     // leave the swarm held away forever.
122     private void OnDisable()
123     {
124         if (occupants.Count > 0 && mode == ZoneMode.SuppressWhileInside)
125             ForEachTarget(swarm => swarm.RemoveSuppressor(this));
126
127         occupants.Clear();
128     }
129
```

`occupants.Remove(other)` records which collider left, and `occupants.Count > 0` preserves the hold until the final occupant exits. `swarm.RemoveSuppressor(this)` identifies the requesting zone. The disabled component also releases its own request, preventing a stale hold. This is explicit ownership and lifecycle design, not a GoF pattern. The same broad idea appears in camera look-suppression keys and phone/menu input claims. PlayerGameState's enum-keyed blocker sets separate categories, but two simultaneous owners using the same enum key are not independently reference-counted.

IntoxicationController provides a common dose and preset blend; IntoxicationAudio and IntoxicationCameraSway apply different outputs. DrunkPreset contains authored settings, and a runtime blend instance protects the source preset. IntoxicationSettings adds persisted accessibility controls and a change event. This is useful separation and a single source for shared parameters, although the controller still combines timing and shader/bloom output. Night-eye sources similarly register per-source exposure or region requests with the adaptation controller. These are concrete component collaborations, not demonstrated interface-based DIP.

Interactions, firewood, and the raccoon

FirewoodLog and Fireplace delegate availability to FirewoodSequence. Their prompts and their actual actions therefore consult the same rule. PileLayout is shared by the carried bundle and fireplace layout, reducing duplicated positioning logic. ShackScareEvent subscribes to pickup events and restores actor materials, control, and lighting when disabled. FirewoodSequence still coordinates carrying animation, instructions, progression, and delivery, so its SRP adherence is partial.

RaccoonController composes GrabbableItem, RaccoonBedKick, RaccoonWindow, and room-light switches, with explicit narrative events for the sequence. The controller also owns awareness, path-graph construction, route search, animation steering, and throwing. Its enum-based phases are a legitimate implementation choice for one encounter, but are not the shared polymorphic State implementation used elsewhere. Door and DoorStory repeat substantial open/close and prompt behavior; shared components or helpers could reduce duplication without creating a deep inheritance hierarchy.

Input, UI, phone, and smaller gameplay components

PlayerInstaller is an actual composition step for player input. InputButton combines bindings with safe default behavior. Footstep and engine audio components listen to gameplay events. Virtual-cursor routing, appearance, affinity, and world-canvas handling are mostly separated into helpers. Direct device reads still exist in some cursor/debug components, so InputManager should not be described as the only input boundary in the entire project.

The opening-drive scripts separate autonomous input, tunnel repositioning, seat-camera behavior, live feed, recording presentation, controls prompt, and final handoff. DrivingControlsPrompt remembers and restores time/audio state, including disable cleanup. OneShotViewCaptureToRawImage restores render targets in finally. MenuPhoneCameraFeed releases its texture resources and validates camera/UI combinations. These are concrete resource-ownership improvements. Coroutine sequencing and DOTween sequences alone do not make these scripts Command objects.

Phone visual helpers use composition and some stable integration/substeps. Camera capture reuses buffers, but gallery item creation remains separate and can grow. Large app classes still mix input, rendering, simulation, and UI. The Signal Words game separates authored card data and card views, while its game controller combines rules, clue scoring, NPC decisions, UI creation, and camera integration. It would benefit from an independent rule/model layer similar to chess.

TreeLeap, IK, tools, and diagnostics

TLMath isolates shared math, TreeRegistry restricts candidate searches, TrunkShape handles geometric queries, TLRig applies a pose description, and TreeShakeSystem owns visual effects and runtime terrain copies. Capture, metrics, and test driving have separate scripts. These are cohesive boundaries. The five partial TreeLeapCreature files remain one class and share private state. Partial declarations improve organization without proving SRP.

IK and spider/skinwalker components separate scanning, pivot orientation, body shake, and limb placement, but include concrete references, duplicated scanning/IK logic, and prototype remnants. Small utilities such as TimedInputBuffer, LightFlicker, VolumetricConeBoundsSetter, FollowTarget, and HouseShadowDistanceZone provide focused behavior rather than special pattern collaborations. HouseShadowDistanceZone restores its own applied shadow-distance override conditionally, preserving a later external override.

Custom drawers and setup scripts keep much authoring behavior outside runtime classes, use validation and Unity Undo, or automate repeatable asset creation. A staged build function is useful automation, not proof of a GoF Builder hierarchy. SystemsTestPanel, TreeLeapMetrics, TreeLeapCapture, and other debug tools aid observation; their existence is not evidence that an automated test suite passed.

Scaffolds and unused abstractions

NpcManager, NpcStateController, several NPC/bear states, PhotoGallery, Date, and SignalWordsVirtualCursorWorldCanvasInput include empty or placeholder behavior. IAudioManager has no implementation/consumer found, and IRotatable has no polymorphic consumer found. ICookable is an empty MonoBehaviour class; Stove uses concrete Food despite a separate ICookableItem interface. These should not be credited as completed pattern or DIP implementations.

20. Focused improvements and verification limits

Preserve the successful boundaries while tightening contracts and ownership.

Priority	Focused change	Why it matters
1	Make Singleton ownership and teardown consistent	Avoid the same-owner destruction path; make scene transitions and service creation predictable.
1	Clarify interaction properties and result semantics	Use read-only query contracts where values are derived; separate configuration from runtime capability.
1	Make state/action cancellation ownership explicit	An operation started by an exited state or skipped node must have a defined completion/cancellation policy.
2	Protect transition/event collections	Private mutation APIs preserve invariants; safe dispatch avoids listener removal invalidating iteration.
2	Extract independent responsibilities from large coordinators	Separate rules, storage, capture resources, input orchestration, and presentation where their reasons to change differ.
2	Inject existing interfaces at integration boundaries	Use the successful car, placement, state-effects, and path-modifier designs as local models; reduce singleton fallbacks gradually.
3	Consolidate repeated mechanics	Share door behavior, repeated spring integration, and similar scanning logic through focused helpers or composition.
3	Remove or label unfinished scaffolds	Make active implementations and intended future extension points clear to maintainers.

One concrete Unity lifetime mismatch

E52 | BearStateMachine is declared as a Unity component

`Final/Assets/0_Scripts/BearEncounter/StateMachine/BearStateMachine.cs`
`Original lines 1-18`

```
using UnityEngine;

namespace Core
{
    public class BearStateMachine : MonoBehaviour
    {
        public StateMachine stateMachine;
        public BearStateData stateData;
        public BearState BearState;
        public BearStateMachine(BearAI Bear)
        {
            SetupStateMachine(Bear);
        }
        void SetupStateMachine(BearAI Bear)
        {
            // State Machine
            stateMachine = new StateMachine();

```

E53 | The owner constructs that type as a plain C# object

`Final/Assets/0_Scripts/BearEncounter/StateMachine/BearAI.cs`
`Original lines 36-47`

```
36          // public CameraController cameraController;
37
38          public SoundID BearWakeSound;
39          public SoundID BearGrowlSound;
40          public SoundID BearAngrySound;
41          private void Awake()
42          {
43              Animator = GetComponent<Animator>();
44              Agent = GetComponent<NavMeshAgent>();
45              BearStateMachine = new BearStateMachine(this);
46              Player.GetComponent<Player>().OnPlayerDeathAnimEvent += OnPlayerDeathAnimEvent;
47              Player.GetComponent<Player>().OnPlayerDeathAnimEventEnded += OnPlayerDeathAnimEventEnd;
```

The source combines `: MonoBehaviour` with `new BearStateMachine(this)`. Unity components have an engine-managed lifecycle, while this constructor call treats the object as an ordinary helper. A plain class fits the current ownership model. Alternatively, a real component would need Unity component creation and explicit initialization. This review identifies the mismatch from source; it does not claim that the encounter was executed or repaired.

Contracts worth documenting explicitly

- State lifecycle: when a state exits, which navigation requests, coroutines, tweens, and player locks must it release? May an old operation still change the actor?
- Story skipping: should already-started actions finish, cancel, or apply a final state? The runner currently allows started actions to continue, so the ordering tradeoff should be intentional.
- Event completion: SceneGroupManager's per-scene notifications are sent after starting async operations, before those operations complete. The group-completed notification is the stronger completion point.
- Interaction: does true mean the action actually happened, was accepted, or was merely handled? Are availability and prompt properties queries or configuration setters?
- Shared data: which fields in ScriptableObjects, trunk profiles, and board state are authored definitions, and which are per-run mutable state? Expose only the mutations consumers should perform.

What was verified

The static review inspected all 388 listed project C# files and cross-referenced the main declarations, implementations, and consumers. During document generation, each code panel was extracted from its declared contiguous line range. A source hash and excerpt hash were recorded, and the ranges were checked again after PDF layout. The final PDF was checked for text extraction, page geometry, and rendered layout. These checks establish document/source consistency, not gameplay correctness.

What remains unverified

No Unity build, Play Mode run, editor validator, automated test suite, save/load recovery scenario, or exhaustive scene/prefab reachability audit was executed. A class can exist and have a consumer in source while still being unused in the shipped scene. Runtime guarantees, threading behavior, cancellation timing, and asset configuration need targeted Unity checks before being certified.

There are also source-level build-separation concerns, including UnityEditor use in GameplayManager outside an editor conditional and a tooling-related import in SignalWordTeamCardView. Their build impact was not tested. Imported vendor code was not reviewed for compliance, and this document makes no assessment of those libraries.

21. Complete script coverage

Every listed file was included in the static review. This is a coverage inventory, not a claim that every file implements a named pattern. Line counts include comments and blank lines. Files may have changed since this snapshot.

. (2)

Script	Lines	Script	Lines
FollowTarget.cs	28	HouseShadowDistanceZone.cs	133

0_Scripts (8)

Script	Lines	Script	Lines
AudioZoneTrigger.cs	17	DialogueEntry.cs	14
GameAudioManager.cs	335	GameEventDefinition.cs	15
GameProgressManager.cs	224	GameProgressStage.cs	21
PlayerGameState.cs	201	ProgressionEventGate.cs	54

0_Scripts/Ambience (1)

Script	Lines	Script	Lines
LightFlicker.cs	77		

0_Scripts/Ambience/Crows (4)

Script	Lines	Script	Lines
CrowAgent.cs	1400	CrowBaker.cs	285
CrowFlock.cs	1552	CrowFlockSettings.cs	264

0_Scripts/Ambience/Moths (3)

Script	Lines	Script	Lines
MothSwarmController.cs	850	MothSwarmSettings.cs	299
MothZoneTrigger.cs	165		

0_Scripts/BearEncounter/StateMachine (14)

Script	Lines	Script	Lines
BearAggresiveAdvanceState.cs	59	BearAI.cs	142
BearBaseState.cs	27	BearChargeState.cs	45
BearDashState.cs	16	BearFakeChargeState.cs	26
BearKillPlayerState.cs	76	BearRoarState.cs	24
BearRunState.cs	23	BearSleepState.cs	39
BearStateData.cs	30	BearStateMachine.cs	48
BearWakeState.cs	34	BearWalkState.cs	86

0_Scripts/ChessMinigame (21)

Script	Lines	Script	Lines
BoardState.cs	270	CapturedPiecesDisplay.cs	176
ChessAI.cs	100	ChessBoardInputBase.cs	352
ChessBoardView.cs	287	ChessDifficultyProfile.cs	155
ChessEngineBridge.cs	156	ChessFenUtility.cs	44
ChessGameManager.cs	474	ChessMoveNotation.cs	202
ChessMoveResult.cs	28	ChessOpeningBook.cs	256
ChessPieceView.cs	16	ChessRules.cs	103
ChessSquare.cs	96	ChessType.cs	135
ChessUI.cs	788	ChessVirtualCursorPlayerInput.cs	45
MoveGenerator.cs	48	NpcChessController.cs	204
PlayerChessInput.cs	28		

0_Scripts/ChessMinigame/Engine (13)

Script	Lines	Script	Lines
Bitboards.cs	317	ChessBrain.cs	192
ChessPieces.cs	57	EngineMove.cs	113
EvalTables.cs	209	Evaluation.cs	449
MoveGen.cs	338	MoveList.cs	75
Position.cs	734	SearchEngine.cs	1024
SearchLimits.cs	87	TranspositionTable.cs	175
Zobrist.cs	45		

0_Scripts/DayNight (4)

Script	Lines	Script	Lines
Observable.cs	54	TimeManager.cs	106
TimeService.cs	43	TimeSettings.cs	10

0_Scripts/Dialogue (1)

Script	Lines	Script	Lines
SpeechBubble.cs	45		

0_Scripts/Enums (1)

Script	Lines	Script	Lines
NpcName.cs	9		

0_Scripts/GameplaySpecific (10)

Script	Lines	Script	Lines
DrivingControlsPrompt.cs	192	IntroCruiseInput.cs	181
IntroPhoneMenuController.cs	368	JumpPrompt.cs	91
MenuCircleCursor.cs	127	MenuPhoneCameraFeed.cs	389
MenuPhoneRecordSequence.cs	368	OneShotViewToCaptureImage.cs	150
PhoneLowerAnimation.cs	53	TunnelLoopWarp.cs	244

0_Scripts/GameplaySpecific/Firewood (5)

Script	Lines	Script	Lines
Fireplace.cs	176	FirewoodLog.cs	107
FirewoodSequence.cs	462	InstructionPrompt.cs	171
ShackScareEvent.cs	575		

0_Scripts/GameState (15)

Script	Lines	Script	Lines
AudioManager.cs	75	BoundaryManager.cs	37
BoundaryPostProcessing.cs	80	BoundaryZone.cs	38
Database.cs	66	DialogueAudioManager.cs	67
GameAssets.cs	48	GameEvent.cs	62
GameEventListener.cs	25	GameEventTrigger.cs	19
GameManager.cs	74	GameplayManager.cs	96
NpcDialogueEnum.cs	7	NpcNameEnum.cs	8
ReferenceHub.cs	18		

0_Scripts/IK (10)

Script	Lines	Script	Lines
IK.cs	104	IK_Feet_Up.cs	123
Player3D.cs	170	PositionFoot.cs	32
ScanArms.cs	67	ScanArmsAdvanced.cs	84
ShakeBody.cs	72	SkinwalkerIK.cs	56
SkinWalkerPivot.cs	52	Spider.cs	375

0_Scripts/Interaction (22)

Script	Lines	Script	Lines
Chair.cs	133	Curtains.cs	94
Door.cs	200	DoorStory.cs	179
DoorTrigger.cs	39	Fridge.cs	147
GrabbableDropZone.cs	146	GrabbableItem.cs	735
InspectableItem.cs	341	InspectableItemManager.cs	76
InteractionSetup.cs	59	IPlacementTarget.cs	21
IRotatable.cs	6	OverlayItem.cs	178
PlacementHint.cs	297	PlateSpot.cs	241
PlateStack.cs	282	PlateTable.cs	120
PlayerBathroom.cs	56	SimpleInteraction.cs	52
SimpleTrigger.cs	58	SlidingWindow.cs	72

0_Scripts/Interaction/IInteractable (1)

Script	Lines	Script	Lines
IInteractable.cs	12		

0_Scripts/Interaction/Pouring (3)

Script	Lines	Script	Lines
LiquidContainer.cs	745	LiquidStream.cs	451
PourableBottle.cs	654		

0_Scripts/Interaction/Stove (4)

Script	Lines	Script	Lines
Food.cs	38	ICookable.cs	16
Stove.cs	87	StoveHandle.cs	99

0_Scripts/Intoxication (7)

Script	Lines	Script	Lines
DrunkPreset.cs	290	GhostLagEcho.cs	256
IntoxicationAudio.cs	203	IntoxicationCameraSway.cs	309
IntoxicationController.cs	591	IntoxicationDebugOverlay.cs	149
IntoxicationSettings.cs	123		

0_Scripts/Intoxication/Editor (1)

Script	Lines	Script	Lines
IntoxicationRendererSetup.cs	150		

0_Scripts/NPC (25)

Script	Lines	Script	Lines
BaseNpcController.cs	450	DiningChair.cs	451
IPathSpeedModifier.cs	14	Npc.cs	83
NpcController.cs	68	NpcConversationController.cs	60
NpcDatabase.cs	81	NpcDialogue.cs	189
NpcDialogueAudios.cs	36	NpcEmotions.cs	108
NpcFollowPlayer.cs	198	NpcGestureController.cs	194
NpcHeadFocus.cs	178	NpcManager.cs	161
NpcPathFollower.cs	552	NpcPathManager.cs	66
NpcStateController.cs	145	NpcTurnInPlace.cs	43
PathGroup.cs	500	PathMarkers.cs	237
PathTrack.cs	422	PathWaitForTarget.cs	359
SitDiningState.cs	626	States.cs	27
TargetDatabase.cs	39		

0_Scripts/NPC/Emily (1)

Script	Lines	Script	Lines
RefrigeratorInteractionState.cs	74		

0_Scripts/NPC/States (9)

Script	Lines	Script	Lines
BaseState.cs	38	IdleState.cs	21
InterruptState.cs	94	MoveState.cs	58
NpcStateData.cs	80	SitState.cs	58
StandState.cs	21	TalkState.cs	37
WaitForPlayerState.cs	146		

0_Scripts/Persistence (5)

Script	Lines	Script	Lines
CheckpointAction.cs	99	PauseMenu.cs	556
PhonePauseMenu.cs	489	SaveData.cs	75
SaveSystem.cs	399		

0_Scripts/Player (8)

Script	Lines	Script	Lines
CameraMode.cs	11	CameraSettings.cs	112
EyeBlink.cs	82	HeadBobSettings.cs	126
IAudioManager.cs	6	InventoryItem.cs	120
PlayerFeelSettings.cs	355	PlayerInventory.cs	399

0_Scripts/Player/PlayerBoundary (4)

Script	Lines	Script	Lines
BoundaryDistortionDriver.cs	221	BoundaryTestStarter.cs	56
PlayerBoundaryManager.cs	108	PlayerBoundaryZone.cs	244

0_Scripts/Player/PlayerBoundary/Editor (1)

Script	Lines	Script	Lines
BoundaryRendererSetup.cs	152		

0_Scripts/Player/PlayerControls (8)

Script	Lines	Script	Lines
CameraControllerNew.cs	1827	Player.cs	123
PlayerFootstepController.cs	209	PlayerInstaller.cs	39
PlayerMovement.cs	1842	PlayerStairFootIK.cs	501
StairSurface.cs	60	ZoomDepthOfField.cs	111

0_Scripts/Player/StateMachineCamera (11)

Script	Lines	Script	Lines
BaseCameraState.cs	45	BoardGameCameraSettings.cs	41
BoardGameCameraState.cs	340	CameraStateChangeEvent.cs	24
DialogueCameraState.cs	145	FreeLookCameraState.cs	22
HeadFollowCameraState.cs	43	LockedCameraState.cs	34
PhoneLookCameraState.cs	43	PlayerCameraStates.cs	135
SittingCameraState.cs	56		

0_Scripts/Player/StateMachinePlayer (13)

Script	Lines	Script	Lines
BasePlayerState.cs	57	DialogueState.cs	162
HaltCameraMovementState.cs	10	HaltLocomotionCameraState.cs	10
HaltMovementState.cs	10	LocomotionState.cs	10
PlayerAnimator.cs	6	PlayerStateController.cs	147
PlayerStateName.cs	10	PlayerStateProfile.cs	57
PlayerStateStack.cs	297	SittingBoardGameState.cs	45
SittingState.cs	49		

0_Scripts/PlayerInput (4)

Script	Lines	Script	Lines
IInitializable.cs	5	InputButton.cs	43
InputManager.cs	414	IPlayerInput.cs	125

0_Scripts/PlayerUI (1)

Script	Lines	Script	Lines
PlayerUI.cs	215		

0_Scripts/Raccoon (4)

Script	Lines	Script	Lines
RaccoonBedKick.cs	109	RaccoonController.cs	859
RaccoonWindow.cs	79	RoomLightSwitch.cs	45

0_Scripts/SceneManagement (3)

Script	Lines	Script	Lines
SceneGroup.cs	30	SceneGroupManager.cs	119
SceneLoader.cs	85		

0_Scripts/Shaders (1)

Script	Lines	Script	Lines
VolumetricConeBoundsSetter.cs	160		

0_Scripts/Signal Words Minigame (5)

Script	Lines	Script	Lines
SignalWordsVirtualCursorWorldCanvasInput.cs	0	SignalWordTeamBoardData.cs	132
SignalWordTeamCardView.cs	286	SignalWordTeamDefinition.cs	59
SignalWordTeamGameController.cs	681		

0_Scripts/Smartphone (34)

Script	Lines	Script	Lines
AppPopup.cs	120	BaseApp.cs	129
ButtonIcon.cs	200	CameraApp.cs	1469
CameraGallery.cs	63	ChatApp.cs	501
Date.cs	18	GalleryApp.cs	117
GalleryImage.cs	11	LightApp.cs	110
LiquidBalloon.cs	193	LiquidFeelSettings.cs	104
LiquidSelectionFill.cs	530	LiquidTMPDeformation.cs	224
Message.cs	102	MusicApp.cs	330
NightVisionLight.cs	85	NotesApp.cs	53
NotesItem.cs	23	NotificationPopup.cs	86
PhoneCursorVisual.cs	391	PhoneFeelSettings.cs	295
PhoneHandCursorArt.cs	157	PhoneHandCursorVisual.cs	215
PhoneScreenParallax.cs	603	PhoneViewFollow.cs	226
PhotoGallery.cs	17	SelectionAffinity.cs	328
Smartphone.cs	713	SmartphoneDragController.cs	944
SmartphoneViewportFitter.cs	335	UINavigationManager.cs	101
VirtualMouseFromMouse.cs	280	WorldCanvasVirtualCursor.cs	1008

0_Scripts/Smartphone/Editor (1)

Script	Lines	Script	Lines
SmartphoneViewportFitterSetup.cs	308		

0_Scripts/StateMachine (6)

Script	Lines	Script	Lines
FuncPredicate.cs	10	IPredicate.cs	6
IState.cs	6	ITransition.cs	4
StateMachine.cs	121	Transition.cs	13

0_Scripts/Story (2)

Script	Lines	Script	Lines
NpcDialogueCinematicAction.cs	91	StoryInteractionHandler.cs	30

0_Scripts/Story/Beats (7)

Script	Lines	Script	Lines
ChoiceAction.cs	355	PerformanceAction.cs	767
PlayerControlAction.cs	305	PropSetupAction.cs	293
StageSetupAction.cs	313	StoryBeatAction.cs	222
WorldStateAction.cs	184		

0_Scripts/Story/Beats/Editor (4)

Script	Lines	Script	Lines
StoryBeatActionEditor.cs	160	StoryNodeEditor.cs	88
StoryTargetDrawer.cs	54	StoryToggleBlockDrawer.cs	208

0_Scripts/Story/ForestBeginning/3) Deer&Fawn (1)

Script	Lines	Script	Lines
DeerMove.cs	103		

0_Scripts/Story/NpcStory (12)

Script	Lines	Script	Lines
BranchingNpcDialogueStoryAction.cs	172	NpcAudioAction.cs	55
NpcDialogueStoryAction.cs	178	NpcGroupWalkAction.cs	91
NpcLookAtStoryAction.cs	110	NpcMoveAction.cs	42
NpcSetInteractStoryNode.cs	36	NpcSetRelativePosRot.cs	66
NpcSetRigStoryAction.cs	183	NpcStateChangeAction.cs	24
NpcStoryAction.cs	70	NpcTimelineEvent.cs	32

0_Scripts/Story/OtherStory (1)

Script	Lines	Script	Lines
SetGameProgressStageAction.cs	17		

0_Scripts/Story/PlayerStory (5)

Script	Lines	Script	Lines
PlayerAudioAction.cs	39	PlayerOrientationStoryAction.cs	71
PlayerStateChangeAction.cs	43	PlayerStoryAction.cs	60
SetPlayerBoundaryAction.cs	35		

0_Scripts/Story/Qte (5)

Script	Lines	Script	Lines
QteBranchNodeAction.cs	67	QteChoiceOptionUI.cs	18
QteChoiceUI.cs	82	QteManager.cs	217
QteStoryAction.cs	87		

0_Scripts/Story/RootStoryAction (7)

Script	Lines	Script	Lines
IStoryInteractionHandler.cs	17	StoryAction.cs	29
StoryDatabase.cs	180	StoryDialogue.cs	99
StoryNode.cs	18	StoryNodeRunner.cs	323
StorySkip.cs	248		

0_Scripts/Story/SmartphoneStory (3)

Script	Lines	Script	Lines
ChatMessageStoryAction.cs	23	JournalUpdateStoryAction.cs	19
TimeStoryAction.cs	29		

0_Scripts/Story/StoryAddons (2)

Script	Lines	Script	Lines
StoryContext.cs	161	WorldDialogueChoiceView.cs	105

0_Scripts/Testing (1)

Script	Lines	Script	Lines
SystemsTestPanel.cs	293		

0_Scripts/Utilities (3)

Script	Lines	Script	Lines
Singleton.cs	55	TakeScreenshot.cs	69
Utils.cs	69		

0_Scripts/Utils (1)

Script	Lines	Script	Lines
TimedInputBuffer.cs	52		

0_Scripts/Vehicle (7)

Script	Lines	Script	Lines
DriverBodyLean.cs	631	DriverSeatCamera.cs	1080
ICarInput.cs	13	SUVCarController.cs	1041
SUVCarInput.cs	28	SUVEngineAudio.cs	371
SUVWheel.cs	109		

0_Scripts/Vehicle/Editor (1)

Script	Lines	Script	Lines
SUVAudioValidation.cs	201		

0_Scripts/VirtualCursor (3)

Script	Lines	Script	Lines
VirtualCursor.cs	137	VirtualCursorUIRaycaster.cs	121
VirtualCursorWorldCanvasUIRaycaster.cs	231		

_TreeLeap/Editor (3)

Script	Lines	Script	Lines
PlaceholderDrawer.cs	22	TreeLeapTestSetup.cs	249
TrunkLibraryBaker.cs	419		

_TreeLeap/Scripts (13)

Script	Lines	Script	Lines
TLMath.cs	186	TLRig.cs	622
TreeLeapCreature.cs	704	TreeLeapCreature.Exit.cs	89
TreeLeapCreature.Ground.cs	324	TreeLeapCreature.Jump.cs	841
TreeLeapCreature.Tree.cs	533	TreeLeapExitTrigger.cs	111
TreeLeapTuning.cs	295	TreeRegistry.cs	158
TreeShakeSystem.cs	376	TrunkLibrary.cs	130
TrunkShape.cs	394		

_TreeLeap/Scripts/Testing (3)

Script	Lines	Script	Lines
TreeLeapCapture.cs	210	TreeLeapMetrics.cs	393
TreeLeapTestDriver.cs	271		

Editor (4)

Script	Lines	Script	Lines
DialogueEntryDrawer.cs	62	NpcTimelineEventDrawer.cs	129
PlayerOmniControllerBuilder.cs	640	PlayModeStateWriter.cs	42

Pesonal_Assets/ALP URP Converter (1)

Script	Lines	Script	Lines
ALP URP Convert.cs	380		

Pesonal_Assets/Custom_Shaders/Night_Vision_Shaders (5)

Script	Lines	Script	Lines
AdaptationController.cs	350	BrightLightExposureZones.cs	159
FlashLightLink.cs	114	NightEyeArtifactRuntime.cs	411
NightEyeRegion.cs	176		